BuyMate Documentation

Team Members

عبد الرحمن محمد توفيق

عمر أشرف مسعد

باسم حاتم صلاح قاسم

عبد الحميد خالد عبد الحميد

محمد علاء الصباحي

Table of contents

Table of Contents

1. Project Planning & Management 3

Overview 3

Objectives 3

Scope 3

Timeline 4

Milestones 5

Deliverables 5

Resource allocation 5

Risks and mitigations 6

KPIs (Key Performance Indicators) 7

2. Literature Review 8
3. Requirements Gathering 9
4. System Analysis & Design 21
5. Implementation (Source Code & Execution) 46
6. Testing & Quality Assurance 47
7. Final Presentation & Reports 47

1. Project Planning & Management

Project Proposal

Overview

BuyMate is an e-commerce platform that enables secure account creation and login, product discovery by categories, cart and wishlist management, coupon-aware checkout, and complete order, payment, and shipment tracking.
The solution is implemented over a normalized relational database with entities such as User, Address, Category, Product, Cart, Wishlist, Order, Payment, Shipment, Coupon, ProductImage, and ProductReview to ensure integrity and scalability.

Objectives

- Provide multi-channel authentication (email/phone credentials and Google OAuth) with optional two-factor authentication for secure access.
- Deliver rich catalog browsing with categories, search, filters, sorting, detailed product pages, multiple images, ratings/reviews, and admin responses to product questions.
- Support complete cart and Wishlist flows with quantity updates, coupon application, and multi-address selection during checkout.
- Orchestrate orders, payments, and shipments with consistent status tracking and correct relational links between entities throughout fulfillment.
- Maintain data integrity and performance using the provided normalized schema, Enums, and foreign keys for all core operations.

Scope

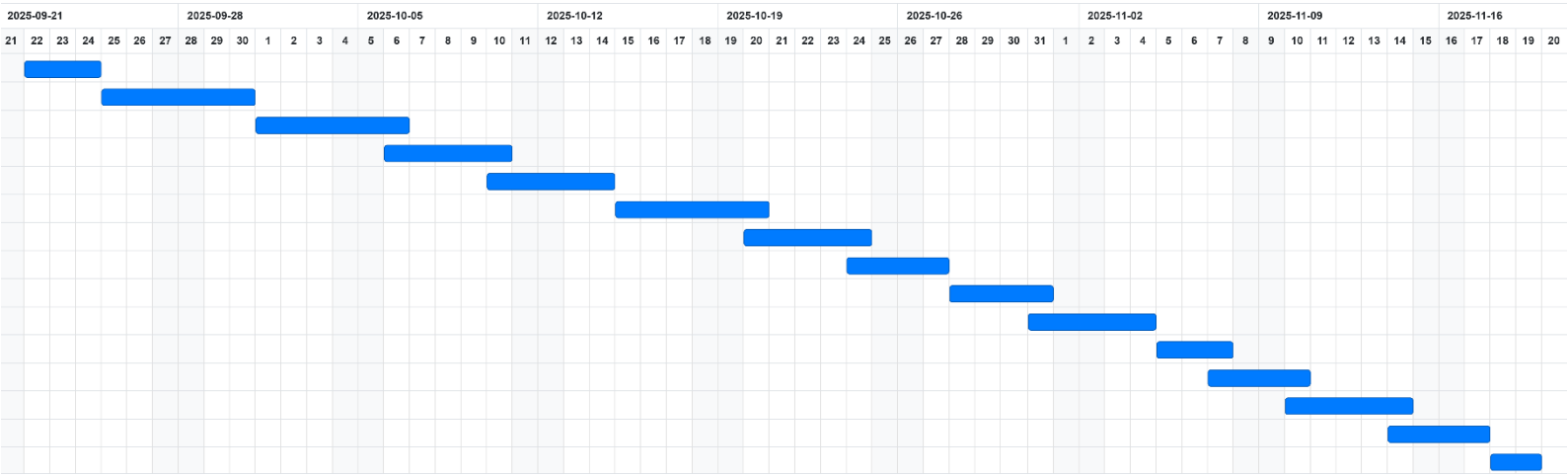
- Accounts and addresses: user registration/login including Google OAuth, optional 2FA, profile editing, and management of multiple delivery addresses.
- Catalog and images: categories and subcategories, search, filters, sorting, detailed product pages with multiple images, stock status, ratings, and Q&A.
- Cart and Wishlist: add, update, and remove items, persist quantities and timestamps, and view totals before checkout.
- Checkout and coupons: apply discount codes, choose saved address and delivery type, and select payment method.
- Orders: create orders with items, capture price at purchase, generate tracking numbers, and view status updates.
- Payments: record payment method, status, amount, transaction details, and link to the related order.
- Shipments: create shipment records per order, set delivery type and dates, costs, and update shipment status.
- Reviews: allow verified users to rate and review purchased products and edit or delete their own reviews.

- Admin basics: managing products, categories, inventory, coupons, and moderate user reviews and questions.
- Technical alignment: implement all features using the normalized ERD/schema with enums and foreign keys to maintain integrity.

Project Plan

Timeline

- 22/09–24/09: Kickoff, finalize proposal and scope, set repo/branching workflow, and publish the schedule.
- 25/09–30/09: Requirements baseline and backlog grooming mapped to entities and user stories.
- 01/10–06/10: Database design finalization, migrations, seed strategy, and integrity rules per schema.
- 06/10–10/10: Accounts and addresses module (auth, OAuth, optional 2FA, profile, multi-address).
- 11/10–14/10: Catalog and images (categories, search, filters, sorting, product details, multi-image).
- 15/10–20/10: Cart and Wishlist (add/update/remove, persist quantities and timestamps).
- 20/10–24/10: Checkout and coupons (apply codes, address and delivery selection, payment option flow).
- 24/10–27/10: Orders core (order creation, items, price-at-purchase, status lifecycle). 1
- 28/10–31/10: Payment integration and recording (methods, status, amount, transaction details).
- 31/10–04/11: Shipments (delivery types and dates, cost, status updates).
- 05/11–07/11: Reviews module for verified purchasers with edit/delete.
- 07/11–10/11: Admin basics (products, categories, inventory, coupons, moderation).
- 10/11–14/11: System and integration testing, bug fixes, performance and security checks.
- 14/11–17/11: Documentation packaging, demo scenarios, and staging readiness.
- 18/11–19/11: Final presentation preparation, release freeze, and sign-off materials.



Milestones

- M1 24/09: Project plan and repository workflow approved.
- M2 04/10: Database schema stable and migrated.
- M3 10/10: Accounts and addresses feature complete.
- M4 23/10: Core shopping flow MVP (catalog → cart → checkout) ready.
- M5 31/10: Payments integrated and verifiable in orders.
- M6 04/11: Shipments integrated with order lifecycle.
- M7 09/11: Reviews and admin basics complete.
- M8 15/11: Implementation tested and ready for packaging.
- M9 19/11: Final presentation and submission complete.

Deliverables

- Project plan artifacts: timeline, milestones, and resource plan stored in the repository per guidelines.
- Database migrations, seed data, and integrity constraints matching the normalized schema.
- Functional modules matching the scope: accounts, catalog, cart/wishlist, checkout/coupons, orders, payments, shipments, reviews, and admin basics.
- Test plan, executed test cases, defect log with resolutions, and QA sign-off.
- Final presentation deck, technical/user documentation, and submission in the designated repository.

Resource allocation

- Project lead: planning, stakeholder sync, risk tracking, and milestone reviews.
- Backend engineer: APIs, business logic, and integration aligned to schema and enums.
- Frontend engineer: UI flows for catalog, cart/checkout, accounts, reviews, and admin.
- Database engineer: migrations, performance tuning, and enforcement of referential integrity.
- QA/tester: test design and execution across modules, regression, and reporting to closure.

Risk Assessment & Mitigation Plan

Risks and mitigations

- Authentication/OAuth/2FA delays or login failures; mitigate by using proven auth/OAuth libraries, staged rollouts, and fallbacks to email/password with comprehensive QA test cases.
- Data integrity issues between Orders, Payments, and Shipments (orphan records, wrong status); mitigate with foreign keys/enums, transactional updates, and end-to-end tests covering state transitions.
- Checkout and coupon miscalculations; mitigate via server-side rule validation, unit tests for discount scenarios, and immutable price-at-purchase capture in Order Items.
- Performance bottlenecks on catalog search/filter and checkout versus KPIs; mitigate with DB indexing, pagination, caching, and load testing prior to release.
- Payment gateway timeouts or errors; mitigate with idempotent retries, clear user messaging, sandbox certification, and fallback flows for pending payments.
- Shipment and order status mismatch; mitigate with a single write path inside transactions, scheduled reconciliation, and alerts on inconsistent states.
- Unauthorized admin actions or role leaks; mitigate with role-based authorization checks on every endpoint and security test cases.
- Review/UGC abuse or spam; mitigate with moderation queue, abuse reporting, rate limiting, and admin responses workflow.
- Schedule compression near final deliverables; mitigate with milestone gates, code freeze before presentation, daily triage, and controlled scope changes.
- Documentation and testing gaps; mitigate by making test evidence and documentation part of the definition of done for each milestone.

KPIs (Key Performance Indicators)

Project Delivery KPIs

- Timeline adherence: Complete all 9 milestones on schedule with zero delays beyond 2 days.
- Feature completeness: Deliver 100% of defined scope (accounts, catalog, cart, checkout, orders, payments, shipments, reviews, admin) by 14/11.
- Documentation quality: All required documents (proposal, plan, roles, risks, KPIs, code, tests, presentation) submitted by deadlines.
- Code coverage: Minimum 80% test coverage for critical modules (authentication, checkout, payment, order processing).

System Performance KPIs

- Response time: Page load time under 2 seconds for catalog browsing, product search, and checkout flows.
- System uptime: Maintain 99% availability during testing phase (10/11–15/11) and final presentation.
- Database performance: All critical queries (search, order creation, payment recording) execute in under 500ms.

Functional Success KPIs

- Authentication success: 95% of login attempts succeed (including OAuth and 2FA flows).
- Checkout completion: 80% of simulated users complete end-to-end checkout without errors.
- Order accuracy: 100% of test orders correctly link to payments, shipments, and apply coupons accurately.
- Data integrity: Zero foreign key violations or orphaned records across User, Order, Payment, and Shipment tables.

User Adoption KPIs

- Catalog usability: Users find target products within 3 search/filter actions.
- Cart conversion: 70% of users who add items to cart proceed to checkout in test scenarios.
- Admin efficiency: Product and coupon management tasks completed in under 2 minutes.

2. Literature Review

- **Feedback & Evaluation** – Lecturer's assessment of the project.
- **Suggested Improvements** – Areas where the project can be enhanced.
- **Final Grading Criteria** – Breakdown of marks based on documentation, implementation, testing, and presentation.

3. Requirements Gathering

3.1 Stakeholder Analysis:

The stakeholder analysis identifies the individuals and groups who have an interest in or are affected by the BuyMate e-commerce system. Each stakeholder's role, interest, and level of influence are described to ensure their needs are considered during the system's design and development.

Stakeholder	Role / Description	Interest / Needs	Influence on the System
Customer (End User)	The primary users who browse, purchase, and review products through the BuyMate platform.	Seamless shopping experience, secure payments, reliable delivery, order tracking, and responsive support.	High — directly impacts system usability, sales, and satisfaction.
Business Admin (Store Admin)	Manages products, categories, inventory, pricing, discounts, and customer interactions through the admin dashboard.	Easy-to-use admin tools, accurate analytics, and secure management capabilities.	High — responsible for day-to-day operations and business success.
Delivery Agent	Handles physical delivery of customer orders and updates delivery status.	Clear order assignments, easy status updates, and accurate delivery details.	Medium — affects customer satisfaction and delivery reliability.
System Owner (Company Management)	The company that owns and operates the BuyMate platform.	Business growth, brand visibility, data-driven decisions, and secure system operations.	High — provides strategic direction and investment.
Development Team	Responsible for designing, building, and maintaining the system.	Clear requirements, maintainable architecture, and efficient collaboration with stakeholders.	High — ensures system functionality, performance, and security.
Cloud Service Provider	Provides hosting and infrastructure (e.g., Azure or AWS) for system deployment.	Stable performance, uptime, and security of hosted services.	Medium — affects system reliability and scalability.
Payment Gateway Provider	Handles online payments through secure transactions.	Reliable, compliant, and easy integration with BuyMate.	Medium — critical for transaction success and trust.

3.2 User Stories & Use Cases:

Use Case 1: User Registration and Login
User Story:
As a customer, I want to register and log in so that I can securely access my account and purchase products.
Actors: Customer, System
Description:
This use case enables customers to create an account using either their email address or phone number and log in using their credentials or Google OAuth.
Preconditions:

- The user must have access to the BuyMate web application.
- The system must be connected to the authentication service (local or Google).

Main Flow:

1. The user clicks “Sign Up.”
2. The system displays the registration form.
3. The user enters full name, email, phone number, password, and address.
4. The system validates input and creates the account.
5. The user can then log in with their credentials or Google account.
6. Upon success, the system redirects to the homepage.

Alternative Flow:

- If email already exists → show “Account already registered.”
- If credentials are invalid → display an error message.

Postconditions:

- A new user account is created and stored in the database.
- User session is initialized.

Use Case 2: Browse and Search Products

User Story:
As a customer, I want to browse and search for products so that I can quickly find what I want.

Actors: Customer, System

Description:

This use case allows the user to browse products by category or search by keyword.

Preconditions:

- The system must have available product data.

Main Flow:

1. The user opens the homepage or category page.
2. The system displays product listings with filters.
3. The user enters a search keyword or applies filters.
4. The system fetches and displays matching products.
5. The user can click on a product to view details.

Alternative Flow:

- If no product matches → show "No results found."

Postconditions:

- Matching products are displayed successfully.

Use Case 3: Add to Cart and Checkout

User Story:
As a customer, I want to add products to my cart and proceed to check out so that I can complete my purchase easily.

Actors: Customer, System, Payment Gateway

Preconditions:

- The user must be logged in.
- Selected items must be available in stock.

Main Flow:

1. The user clicks "Add to Cart" on a product.
2. The system updates the cart with the selected item.
3. The user opens the cart page.
4. The user reviews items and clicks "Proceed to Check out."
5. The system displays payment and delivery options.
6. The user selects payment method (Credit Card, Wallet, or Cash on Delivery).
7. The system processes payment through the gateway.
8. The order is confirmed.

Alternative Flow:

- If payment fails → display "Payment failed, please try again."

- If an item goes out of stock → show “Item unavailable.”

Postconditions:

- Order is created and stored.
- Confirmation email/SMS is sent to the customer.

Use Case 4: Track Order

User Story:
As a customer, I want to track my order so that I can know the delivery status.

Actors: Customer, System, Delivery Agent

Preconditions:

- The order must exist and have a tracking ID.

Main Flow:

1. The user opens “My Orders.”
2. The system retrieves all recent orders.
3. The user selects an order to view details.
4. The system displays status (Processing, Shipped, Out for Delivery, Delivered).

Alternative Flow:

- If the order is delayed, → system shows the expected delivery date.

Postconditions:

- The user sees the latest order progress.

Use Case 5: Manage Wishlist

User Story:
As a customer, I want to save products to my Wishlist so that I can buy them later.

Actors: Customer, System

Preconditions:

- The user must be logged in.

Main Flow:

1. The user clicks the heart icon on a product.
2. The system adds the product to the user’s Wishlist.
3. The user can view their Wishlist anytime.

Alternative Flow:

- If the product is already in the Wishlist, → system notifies the user.

Postconditions:

- The product is saved on the user's Wishlist table.

Use Case 6: Product Management (Admin)

User Story:
As an admin, I want to add, update, and delete products so that I can maintain accurate product listings.

Actors: Admin, System

Preconditions:

- The admin must be logged in.

Main Flow:

1. The admin logs into the dashboard.
2. Selects "Manage Products."
3. Clicks "Add Product."
4. Enters title, description, price, stock, and uploads images.
5. Saves the product.
6. System updates the database and product listing.

Alternative Flow:

- If mandatory fields are missing, → system prompts to complete.

Postconditions:

- Product list updated in the system.

Use Case 7: Manage Discounts & Coupons (Admin)

User Story:
As an admin, I want to create discount codes and promotions so that I can encourage sales.

Actors: Admin, System

Preconditions:

- Admin is authenticated.

Main Flow:

1. The admin opens "Discounts & Promotions."
2. Clicks "Create Coupon."
3. Sets code, percentage, validity date, and target products.
4. System saves and activates the coupon.

Alternative Flow:

- If code already exists → system notifies admin.

Postconditions:

- Coupon stored and ready for use at checkout.

Use Case 8: Manage Delivery & Status (Delivery Agent)

User Story:
As a delivery agent, I want to view and update my assigned deliveries so that customers can track their orders.

Actors: Delivery Agent, System

Preconditions:

- The agent must be authenticated.
- The system must have active delivery assignments.

Main Flow:

1. The agent logs into the delivery portal.
2. Views assigned deliveries with customer details.
3. Marks “Picked Up,” then “Out for Delivery.”
4. Upon completion, marks “Delivered.”

Alternative Flow:

- If delivery fails → mark as “Returned” with a reason.

Postconditions:

- System updates delivery status for customer visibility.

Use Case 9: Sales Analytics & Dashboard (Admin)

User Story:
As an admin, I want to view sales analytics so that I can make data-driven business decisions.

Actors: Admin, System

Preconditions:

- Admin must be logged in.

Main Flow:

1. The admin navigates to “Dashboard.”
2. System displays charts: sales by category, region, profit margins, etc.
3. Admin filters by time period.

Alternative Flow:

- If no sales data exists, → system displays “No data available.”

Postconditions:

- Dashboard displays updated statistics.

Use Case 10: Forgot Password and Reset

User Story:
As a customer, I want to reset my password if I forget it so that I can regain access to my account.

Actors: Customer, System, Email Service

Description:
Allows users to reset forgotten passwords securely via an emailed link or OTP.

Preconditions:

- User account must exist.

Main Flow:

1. The user clicks “Forgot Password.”
2. The system asks for a registered email/phone.
3. The user provides input.
4. The system sends a reset link or OTP.
5. The user enters a new password and confirms it.
6. The system updates the credentials.

Alternative Flow:

- If email not found → show “Account not found.”
- If OTP expired → prompt to resend.

Postconditions:

- Password updated successfully.
- Old sessions invalidated.

Use Case 11: Manage Multiple Delivery Addresses

User Story:
As a customer, I want to manage multiple delivery addresses so that I can choose where my order should be delivered.

Actors: Customer, System

Description:
This use case enables customers to add, edit, and delete multiple saved delivery addresses in their profile and select one during checkout.

Preconditions:

- The customer must be logged in.
- The system must have access to customer profile data.

Main Flow:

1. The customer navigates to “My Account” → “Manage Addresses.”
2. The system displays a list of existing saved addresses (if any).
3. The customer clicks “Add New Address.”

4. The customer enters address details (name, street, city, region, postal code, phone number).
5. The system validates and saves the new address.
6. During checkout, the customer selects one saved address for delivery.

Alternative Flow:

- If the address form is incomplete, → system displays validation messages.
- If a duplicate address is detected, → system prompts "Address already exists."

Postconditions:

- Address saved and available for selection during checkout.
- Customer can modify or delete saved addresses at any time

Use Case 12: View Detailed Product Information

User Story:
As a customer, I want to view detailed product information (images, description, stock) so that I can make an informed purchase decision.

Actors: Customer, System

Description:
This use case enables customers to view full product details, including images, description, specifications, reviews, and stock availability.

Preconditions:

- The product must exist and be active in the catalog.

Main Flow:

1. The customer clicks on a product from the homepage or search results.
2. The system retrieves product details from the database.
3. The system displays images, title, price, stock status, description, reviews, and specifications.
4. The customer can zoom into images or view additional photos.
5. The system recommends related products below the main section.

Alternative Flow:

- If the product is out of stock, → system displays "Out of stock."
- If product removed or unavailable, → system redirects to "Product not found" page.

Postconditions:

- Customer successfully views product details.

Use Case 13: Apply Discount Codes During Checkout

User Story:

As a customer, I want to apply discount codes during checkout so that I can save money.

Actors: Customer, System, Payment Gateway

Description:
Allows customers to apply valid discount or coupon codes at checkout for eligible products.

Preconditions:

- The customer must be logged in.
- A valid and active coupon code must exist.
- The cart must contain eligible items.

Main Flow:

1. The customer proceeds to the checkout page.
2. The system displays an input field labeled “Apply Discount Code.”
3. The customer enters a coupon code and clicks “Apply.”
4. The system validates the code against the database.
5. If valid, the system applies the discount to the total amount.
6. The new total is displayed to the customer.

Alternative Flow:

- If the code is invalid or expired, → system shows “Invalid or expired coupon.”
- If the coupon does not apply to selected items, → system displays “Coupon not valid for these products.”

Postconditions:

- Discount applied to the current transaction.
- System logs coupon usage.

3.3 Functional Requirements:

3.3.1 Users

Account & Authentication

- Users can register a new account using their username, email, phone number, full name, and address, or log in using Google (OAuth).
- Users can enable Two-Factor Authentication (2FA) for enhanced account security.
- Users can log in using email/phone and password, or their linked Google account.
- Users can edit their profile information, including name, email, phone, addresses, and profile image.
- Users can manage multiple delivery addresses.

Product Browsing & Search

- Users can browse products by categories and subcategories.
- Users can search for products by name, keyword, or apply advanced filters.
- Sorting options include price, name, rating, discount, and newest arrivals.
- Users can view detailed product information, including description, price, stock availability, ratings, multiple images, and discounts.
- Users can post questions on product pages; administrators can respond to inquiries.

Cart & Checkout

- Users can add or remove products from their cart.
- Users can update product quantities within the cart.
- Users can review the cart contents before proceeding to check out.
- Users can apply valid coupon or discount codes.
- Users can select a preferred payment method (Credit Card, Wallet, or Cash on Delivery).
- Users can choose from the following delivery types:
    - Standard Delivery (default, cost-effective option).
    - Express/Same-Day Delivery (extra fee, limited locations).
    - Scheduled Delivery (choose a specific date and time slot).
- Users can select one of their saved addresses for order delivery.

Order Management

- Users receive an order confirmation after completing checkout.
- Users can track orders using a tracking number to view their status (Processing, Shipped, Out for Delivery, Delivered).
- For scheduled deliveries, users can reschedule within a defined limit if the order has not been assigned to a delivery agent.
- For Cash on Delivery orders, users pay the delivery agent upon receiving the package.

Wishlist & Notifications

- Users can add or remove products from their Wishlist.
- Users receive notifications when a wish-listed product:
    - Experiences a price drop.
    - Receive a discount.
    - Returns to stock after being unavailable.

Reviews & Ratings

- Users can write reviews and rate products they have purchased.
- Users can edit or delete their own reviews.
- Users can report inappropriate or abusive reviews.

History & Analytics

- Users can view their purchase history and previous orders.
- Users can access personal spending analytics dashboards, including:
    - Total spending by month.
    - Spending breakdown by product category.
    - Average order value.

3.3.2 Business Admin (Store Admin)

Business administrators manage the e-commerce system, products, users, and analytics through an admin interface.

Product & Category Management

- Add, edit, and delete products, including title, description, price, stock, and images.
- Create and manage product categories and subcategories.
- Monitor and manage inventory levels and stock updates.

Discounts & Promotions

- Apply discounts to specific products, categories, or subcategories.
- Create bundle offers (e.g., Buy 2 Get 1 Free).
- Generate and manage coupon codes.

User & Review Management

- Approve or reject submitted product reviews.
- Respond to flagged questions or inappropriate reviews.
- Manage user accounts, including the ability to block or unblock users.

Delivery & Payment Configuration

- Configure available delivery types (Standard, Express, Scheduled).
- Set delivery fees and availability based on region or product.
- Define Cash on Delivery availability.
- Manage supported payment gateways, including credit cards, PayPal, and digital wallets.

Analytics & Dashboards

- View sales analytics, including:
    - Sales by product, category, and region.
    - Revenue growth and profit margins.
- View delivery analytics, including usage rates for Standard, Express, and Scheduled deliveries.

3.3.3 Delivery Agent

Delivery agents:

- manage physical order fulfillment through a dedicated delivery interface.

- Can log in to the delivery portal and view assigned deliveries.
- Can view delivery details, including customer name, address, contact information, and delivery type or time slot.
- Can update delivery status throughout the process (Picked Up → Out for Delivery → Delivered).

For Cash on Delivery orders, the user must confirm delivery completion only after cash has been received.

3.4 Non-Functional Requirements:

The non-functional requirements define the quality attributes and operational characteristics that ensure the BuyMate system performs reliably, securely, and efficiently.

3.4.1 Performance Requirements

- The system shall load all main pages (Home, Product Listing, Cart, Checkout) within 3 seconds under normal network conditions.
- The system should support at least 200 concurrent users without performance degradation.
- Database queries shall be optimized to ensure response times below 2 seconds for search and filtering operations.
- The system should utilize caching mechanisms to improve performance and reduce server load.

3.4.2 Reliability & Availability

- The system shall maintain an uptime of at least 99.5% during business hours.
- Automatic recovery and error logging shall be implemented to handle unexpected crashes or server issues.
- All transactions (orders, payments, user updates) shall ensure atomicity and consistency through transactional control.
- Backup procedures should be run daily to prevent data loss.

3.4.3 Scalability

- The system architecture shall allow horizontal and vertical scaling to handle increased traffic and data growth.
- Cloud deployment shall enable easy addition of resources (servers, storage, bandwidth) without downtime.

3.4.4 Security

- All communications between client and server shall be secured using HTTPS (SSL/TLS).
- User passwords shall be hashed and salted before storage using industry-standard encryption.
- Role-based access control (RBAC) shall ensure restricted access for different roles (Users, Admins, Delivery Agents).
- The system shall comply with OWASP security standards to protect against common vulnerabilities (SQL Injection, XSS, CSRF).
- Sensitive data (e.g., payment details) shall not be stored on the platform and shall be handled by trusted third-party gateways.

3.4.5 Usability

- The interface shall be intuitive, consistent, and responsive across all modern browsers and screen sizes.
- Buttons, menus, and input forms shall follow a consistent visual design for clarity and ease of use.
- The system shall provide multi-language and accessibility support (WCAG compliance where applicable).
- New users should be able to complete their first purchase with minimal guidance.

3.4.6 Maintainability

- The system shall follow modular MVC architecture to facilitate updates and future feature expansion.
- The codebase shall adhere to .NET and C# coding standards with meaningful comments and documentation.
- Logging and monitoring tools shall assist in diagnosing issues quickly.
- Version control using GitHub shall ensure smooth collaboration and change tracking.

3.4.7 Portability

- The application shall be deployable on multiple cloud environments (Azure, AWS, or any Windows-based server).
- The codebase shall remain platform-independent and compatible with future .NET versions.

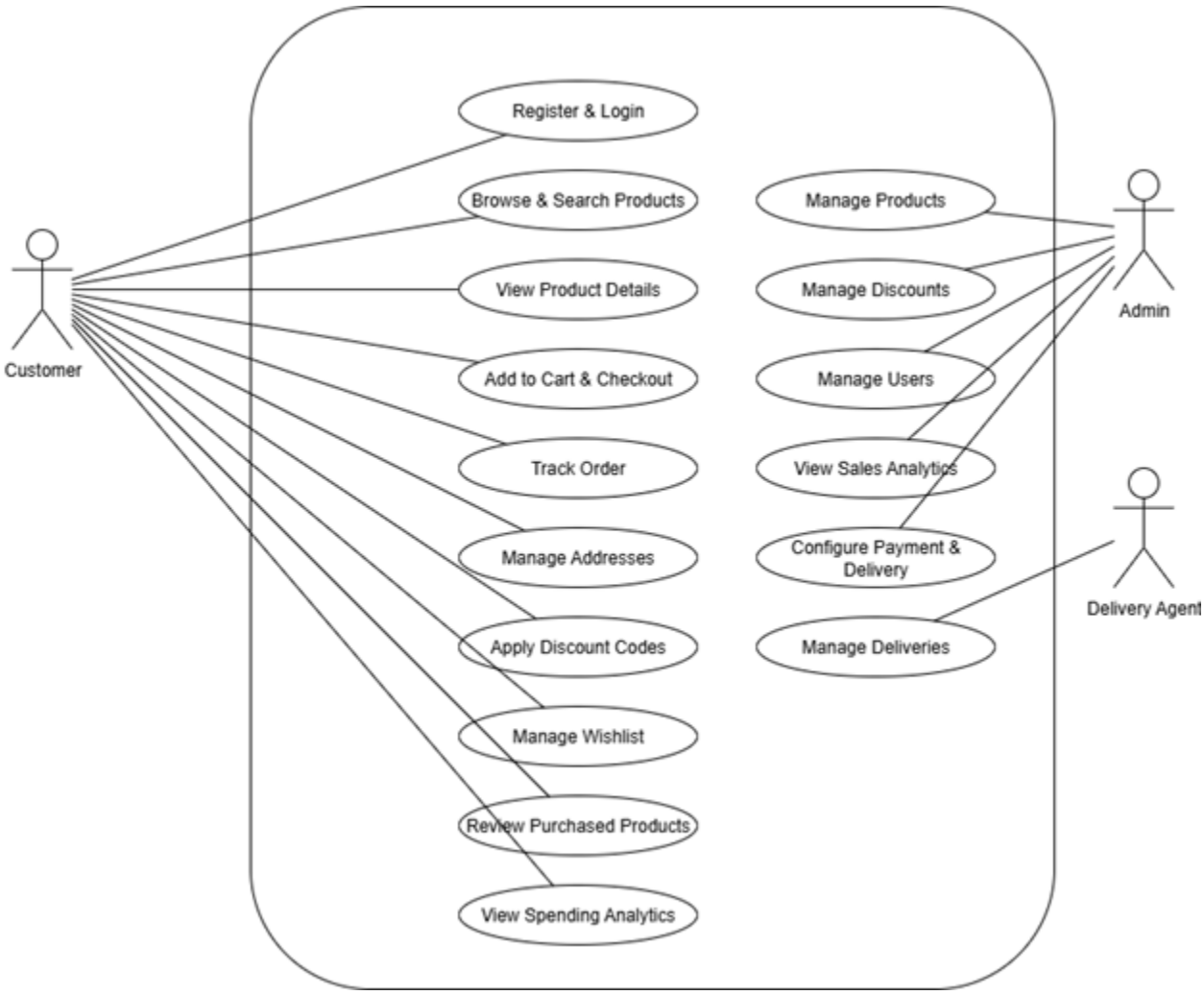
3.4.8 Data Integrity & Backup

- All system data (users, products, orders) shall be stored in a relational database with proper constraints and normalization.
- Automatic daily backups shall be configured on the cloud environment.
- Recovery mechanisms shall allow data restoration within 24 hours in case of failure.

4. System Analysis & Design

i. Problem Statement & Objectives

- **Use Case Diagram:**



The BuyMate Use Case Diagram illustrates the key interactions between the system and its main actors — Customer, Admin, and Delivery Agent.
Customers can browse and purchase products, manage orders, apply coupons, review purchases, and track spending. Admins handle product management, inventory, promotions, and analytics. Delivery agents manage assigned orders and update delivery statuses.

- **Functional & Non-Functional Requirements**

Functional Requirements

The system provides all essential functionalities of an e-commerce web application, including:

- **User Management:** Customer registration, authentication, and profile management.

- **Product Management:** Adding, editing, deleting, and categorizing products.
- **Shopping & Checkout:** Product browsing, search, cart, checkout, and order processing.
- **Promotions:** Managing coupons, discounts, and bundle offers.
- **Order & Delivery Management:** Order tracking, delivery status updates, and payment handling.
- **Analytics & Reporting:** Viewing sales, revenue, and performance metrics.
- **Notifications:** Email and SMS alerts for order confirmations, updates, and promotions.

Non-Functional Requirements

- **Performance:** The system should handle multiple concurrent users with minimal response time (<3 seconds for typical operations).
- **Security:** Implements secure authentication, HTTPS encryption, and data validation.
- **Scalability:** Built on .NET Core MVC and deployable to the cloud for high availability.
- **Usability:** Provides an intuitive and responsive interface across devices.
- **Maintainability:** Modular MVC structure enables easy updates and enhancements.
- **Reliability:** Automatic backups and robust error handling minimize downtime.
- **Compatibility:** Supports all major web browsers and adapts to desktop and mobile screens.

• Software Architecture

The BuyMate E-Commerce System is built using the **ASP.NET Core MVC (Model-View-Controller)** architecture, which provides a clean separation of concerns and promotes scalability, maintainability, and reusability.
The architecture is divided into three main components:

1. **Model Layer:**
   - Represents the business logic and data access layer of the system.
   - Manages entities such as Users, Products, Orders, Cart, Categories, Reviews, and Payments.
   - Communicates with the underlying SQL Server database using Entity Framework Core (ORM) for data persistence and retrieval.
2. **View Layer:**
   - Handles the user interface and presentation logic.
   - Implements responsive, interactive web pages using Razor Views, HTML5, CSS3, Bootstrap, and JavaScript.
   - Provides separate interfaces for Customers, Admins, and Delivery Agents to ensure usability and role-based access.
3. **Controller Layer:**
   - Acts as the intermediary between the View and Model layers.

- Handles HTTP requests and responses, processes user inputs, applies validation, and calls the appropriate business logic in the Model layer.
- Returns results to the View layer in the form of data or rendered views.

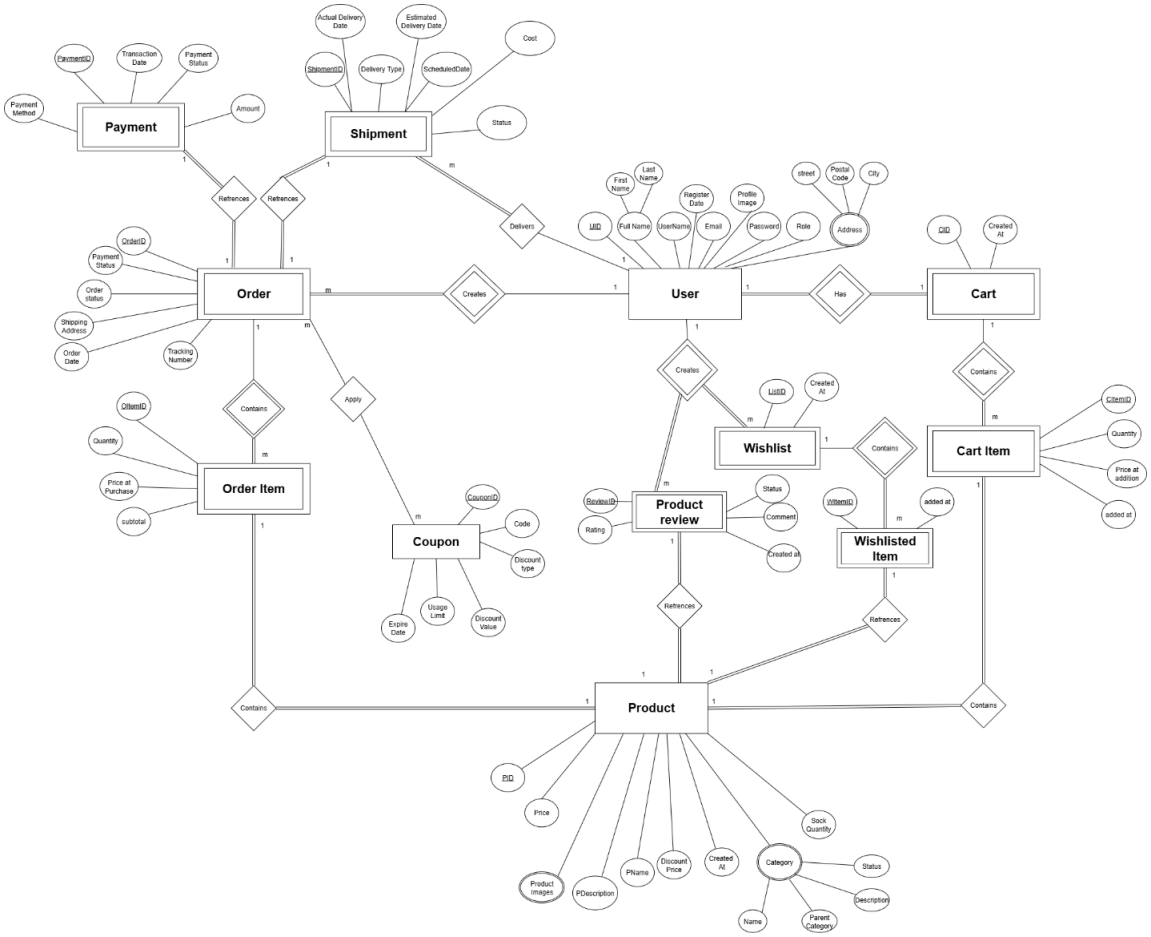
System Components and Interactions

The BuyMate architecture includes several interacting subsystems and modules:

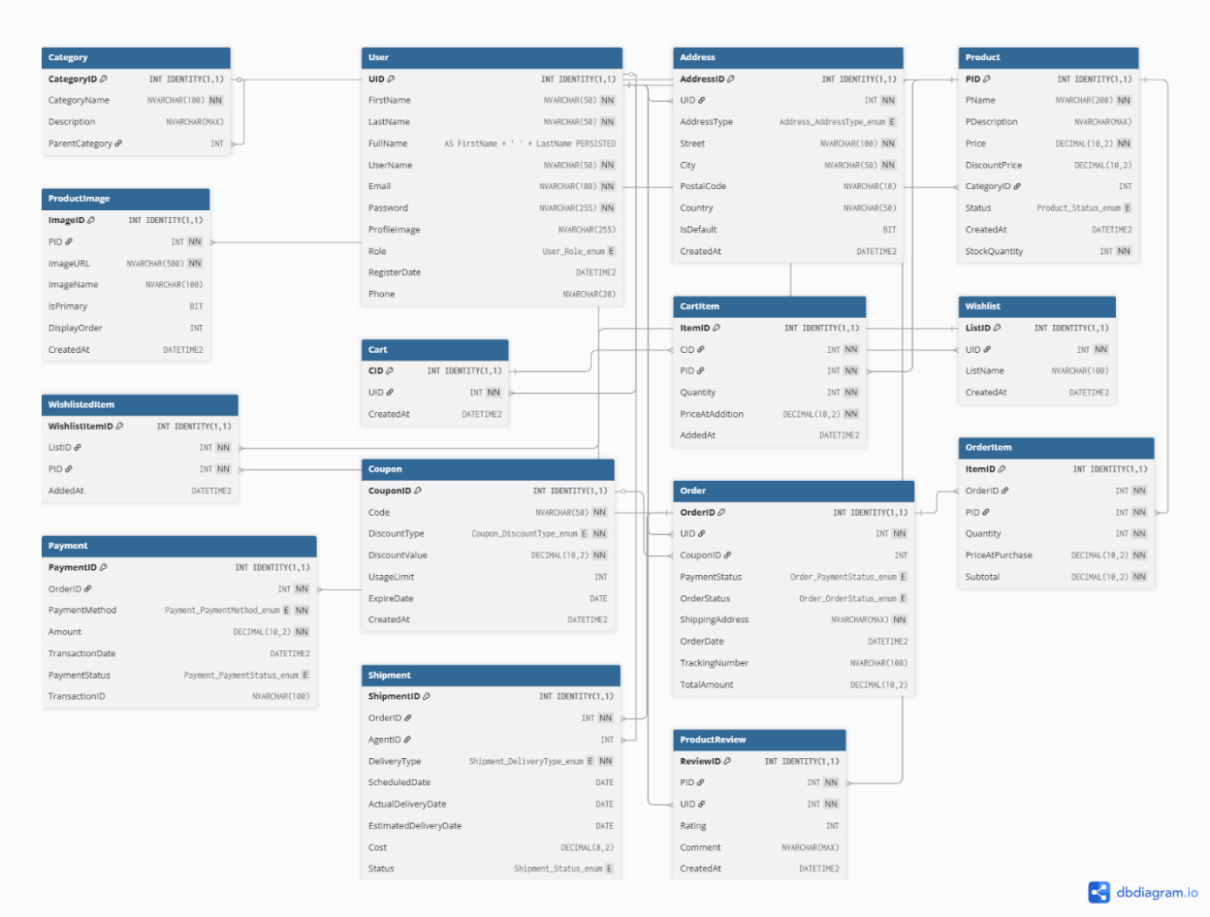
- **Authentication & Authorization Module:**
  Handles user registration, login, and role-based access control using ASP.NET Identity.
- **Product & Inventory Module:**
  Allows the admin to manage product listings, categories, stock levels, and pricing.
- **Shopping & Checkout Module:**
  Manages the shopping cart, checkout flow, discount application, and order processing.
- **Payment Gateway Integration:**
  Supports multiple payment methods (e.g., Credit Card, PayPal, and Cash on Delivery) through API integration.
- **Order & Delivery Management Module:**
  Tracks order progress, delivery scheduling, and delivery status updates by agents.
- **Analytics & Reporting Module:**
  Provides visual dashboards for sales performance, revenue, and order metrics.
- **Notification Service:**
  Sends real-time alerts via **email/SMS** for order confirmations, shipping updates, and promotions.

ii. Database Design & Data Modeling

- **ER Diagram (Entity-Relationship Diagram):**



• **Logical & Physical Schema:**



iii. Data Flow & System Behavior

• DFD (Data Flow Diagram) – Context-level and detailed levels showing how data moves through the system.

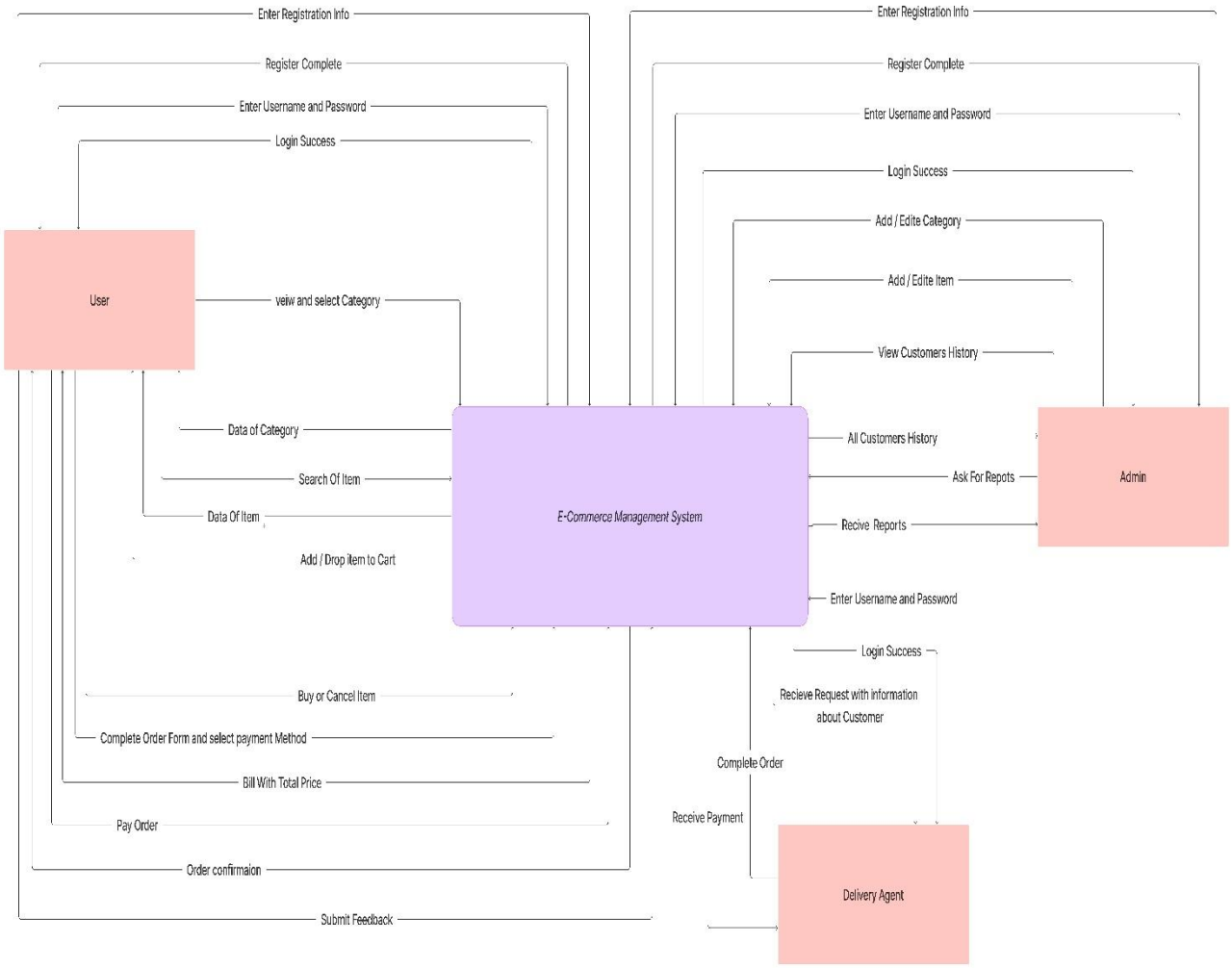
• Sequence Diagrams – Process flow representation of key interactions between components.

• Activity Diagram – Visualizing the workflow of processes or user actions within the system.

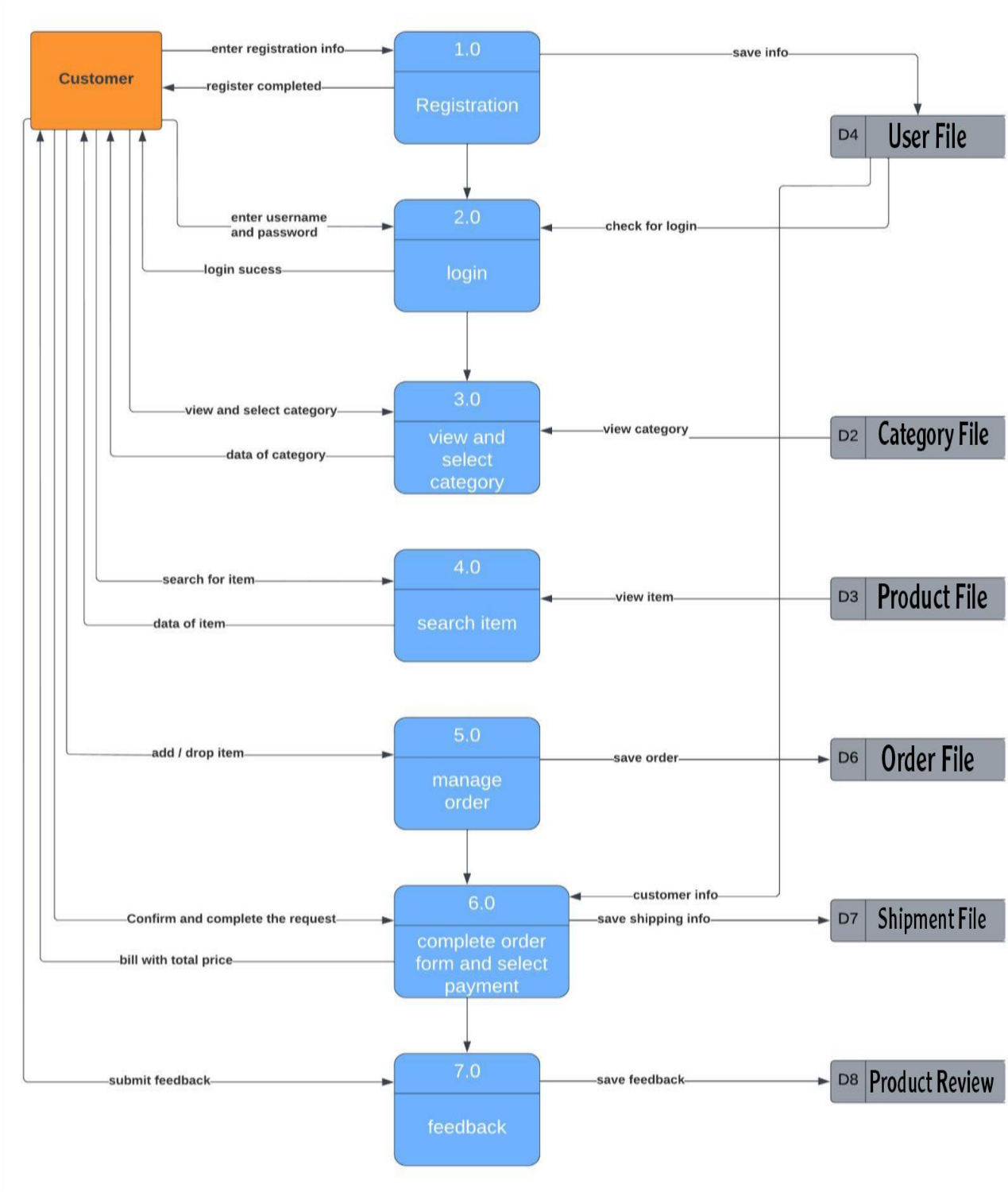
• State Diagram – Represents different states of an object and how it transitions between them.

• Class Diagram – Defines the structure of the system by showing classes, attributes, methods, and relationships.

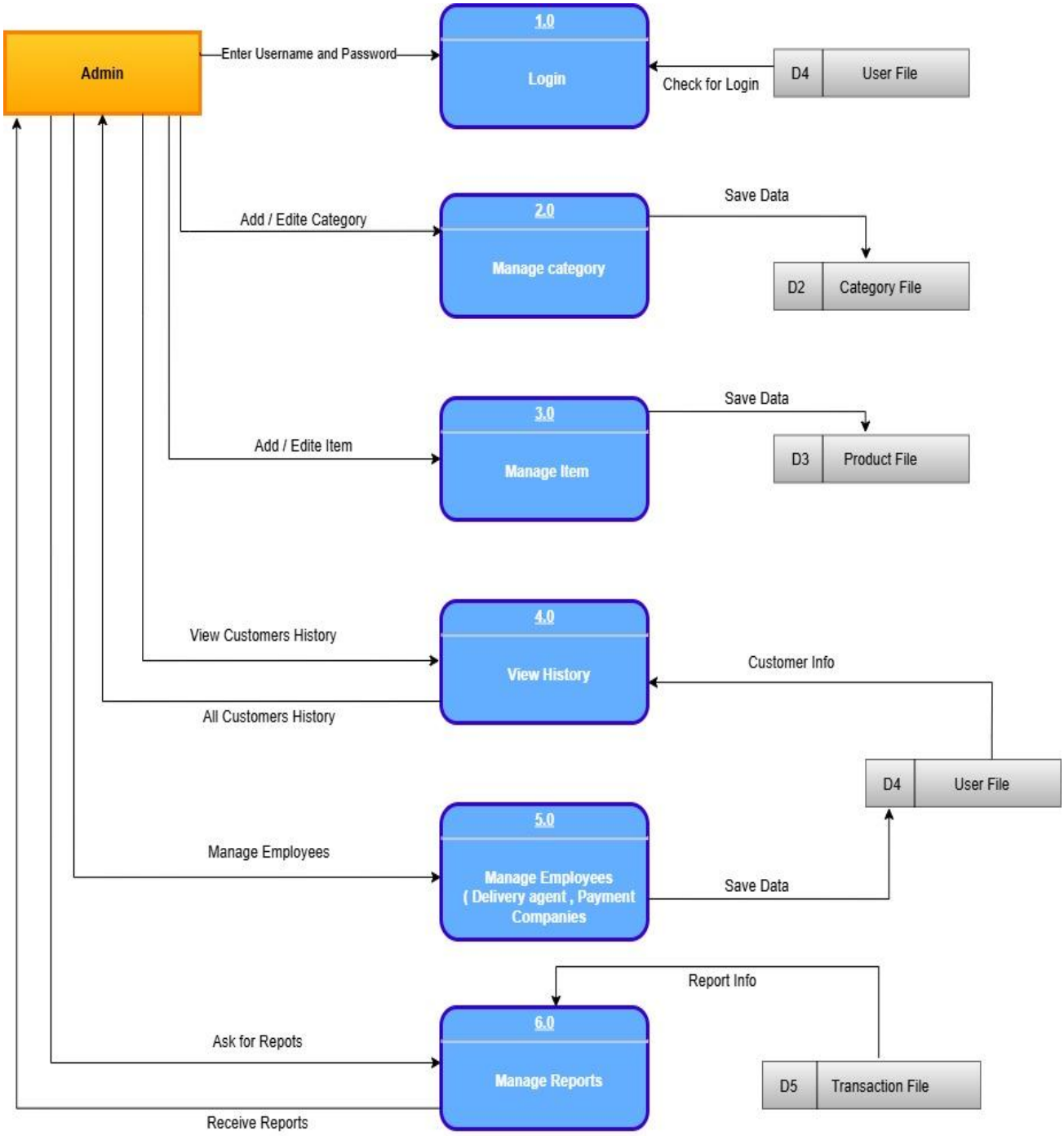
Context Diagram



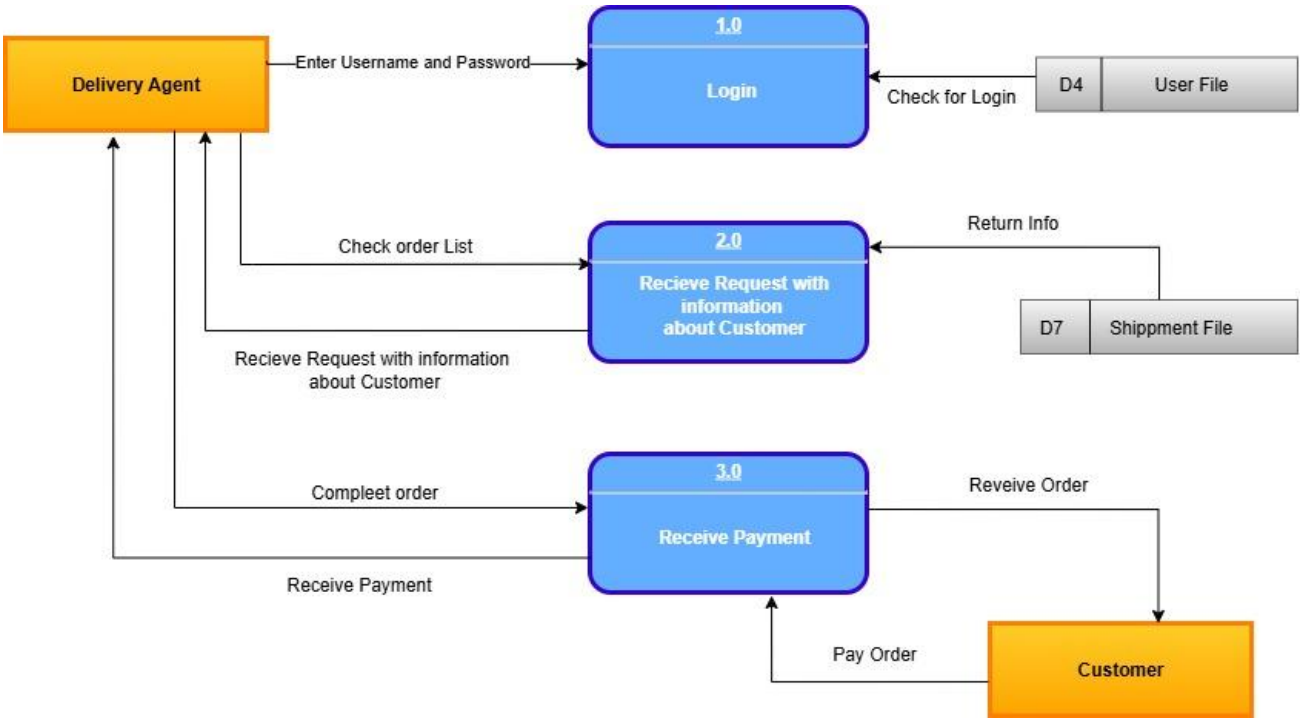
Customer DFD



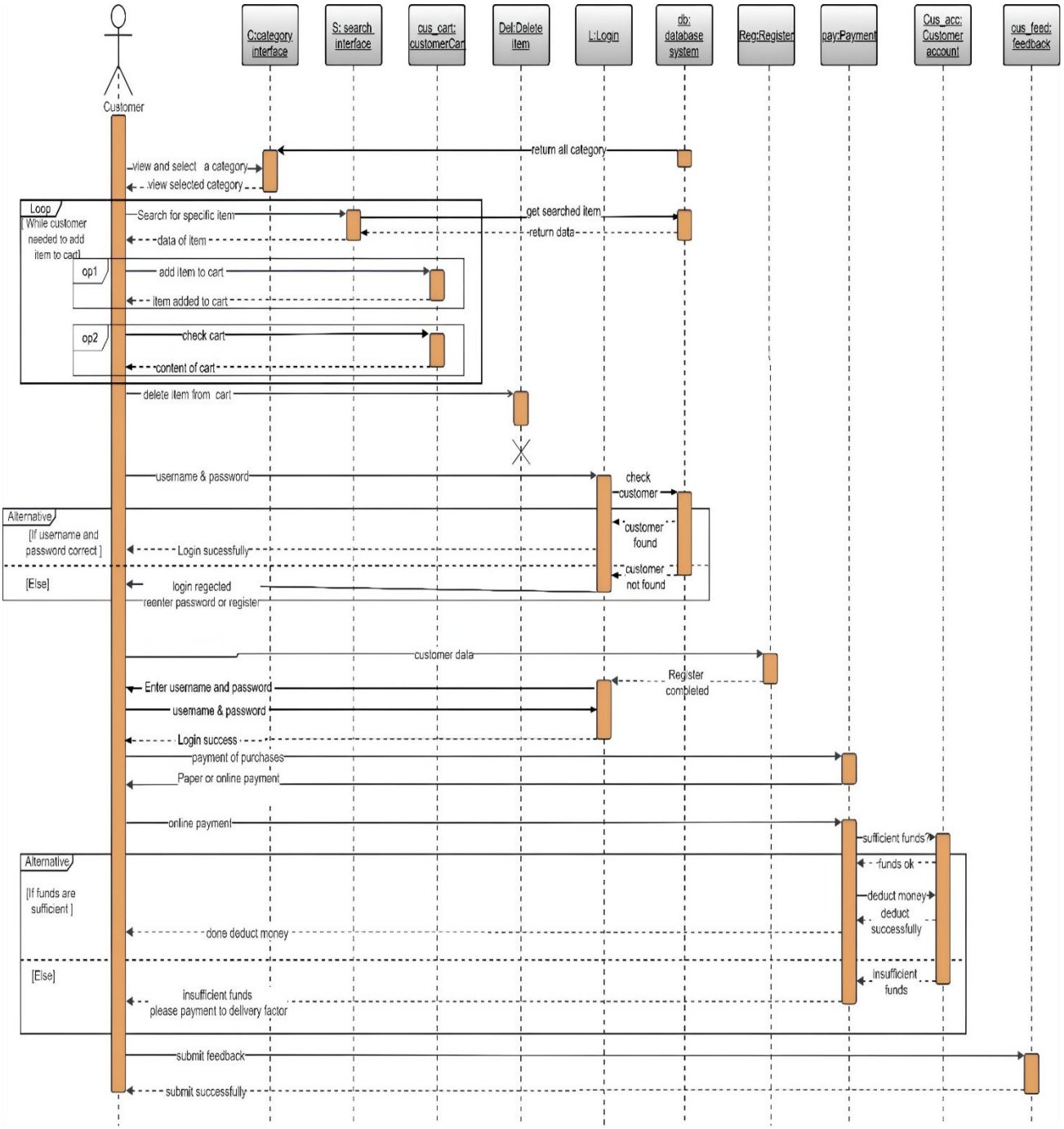
Admin DFD


Admin
Enter Username and Password
1.0
Login
Check for Login
D4
User File
Add / Edite Category
2.0
Manage category
Save Data
D2
Category File
Add / Edite Item
3.0
Manage Item
Save Data
D3
Product File
View Customers History
4.0
View History
Customer Info
All Customers History
D4
User File
Manage Employees
5.0
Manage Employees (Delivery agent , Payment Companies
Save Data
Report Info
Ask for Repots
6.0
Manage Reports
D5
Transaction File
Receive Reports

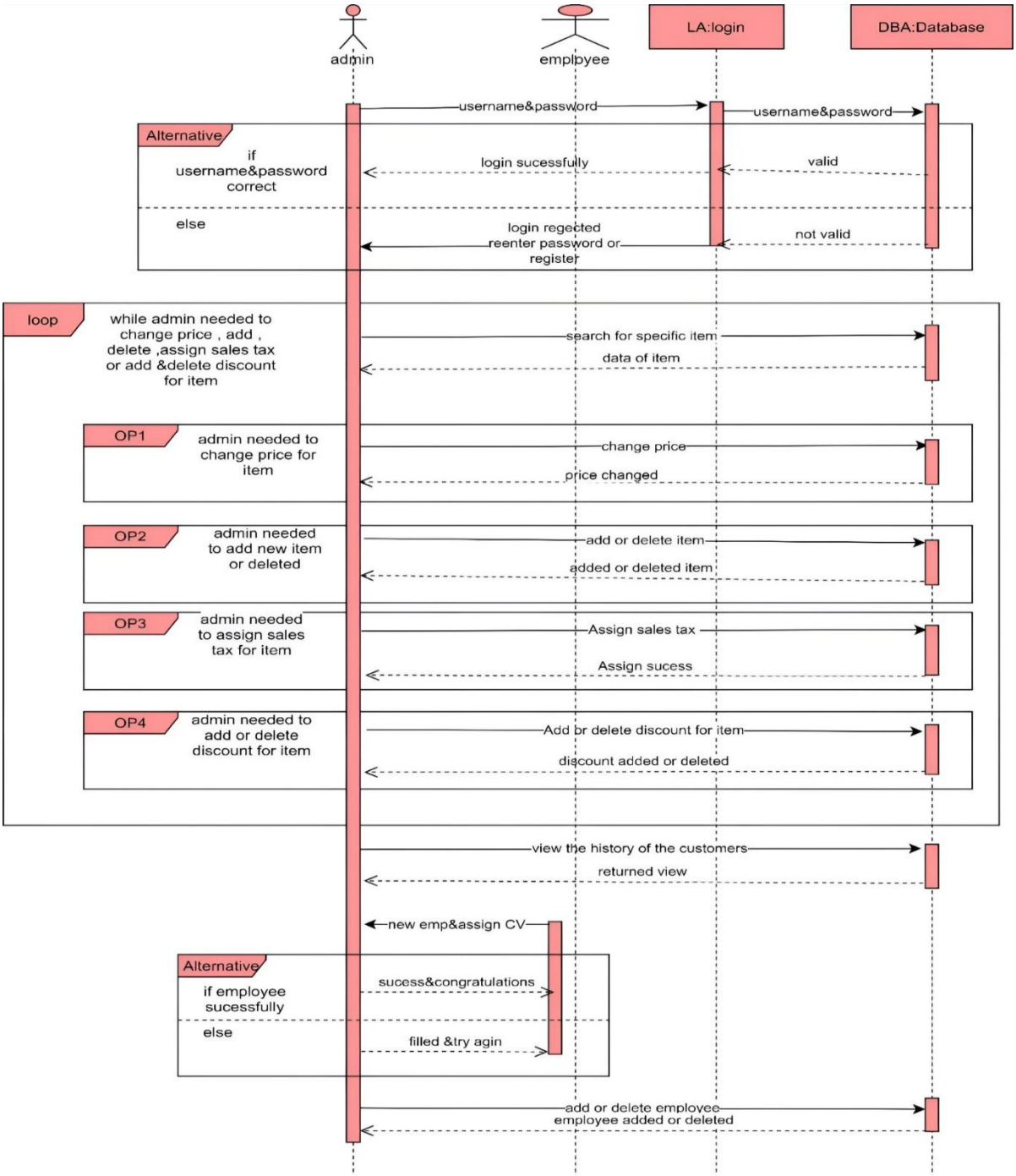
Delivery Agent DFD



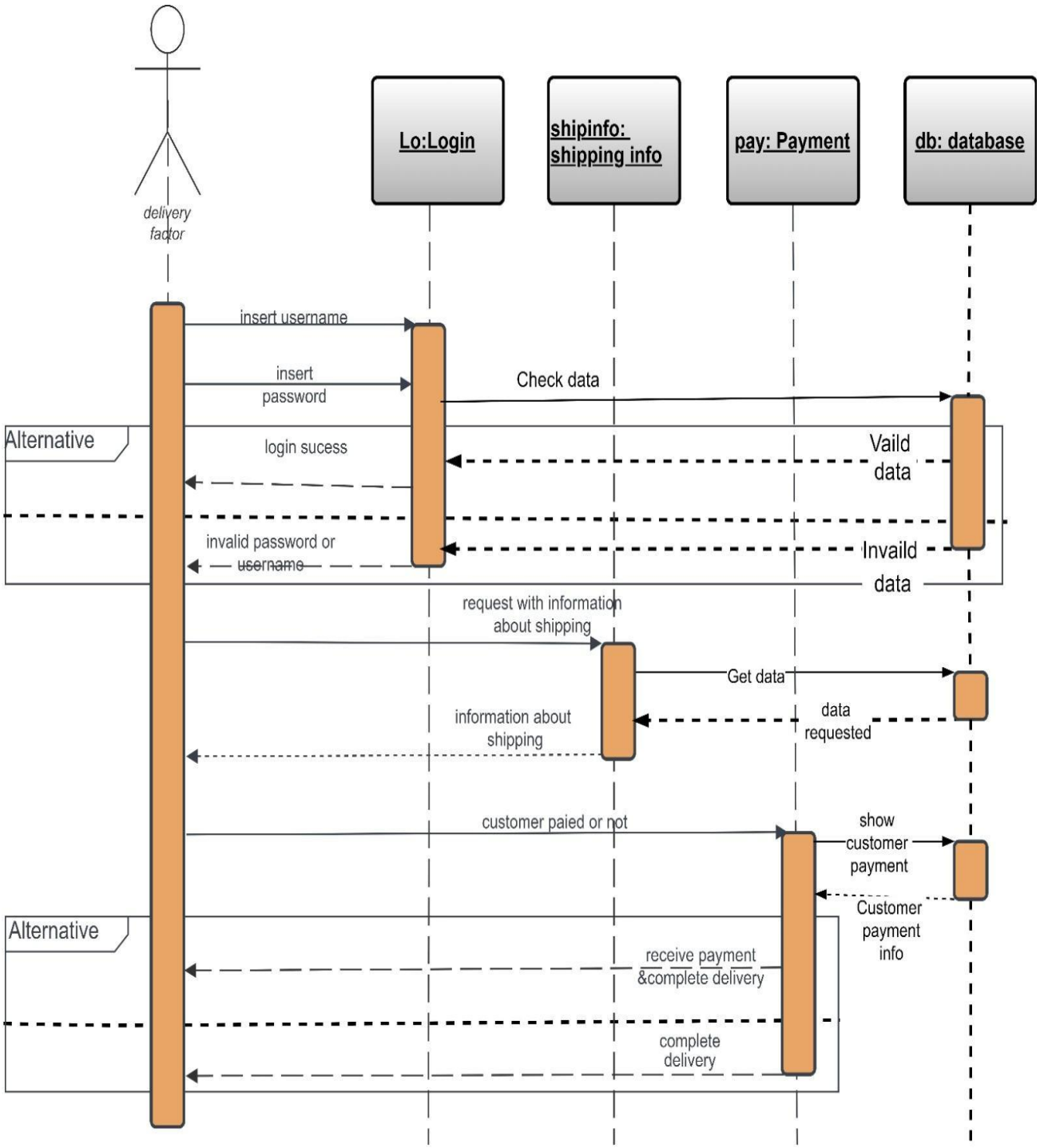
Customer Sequence



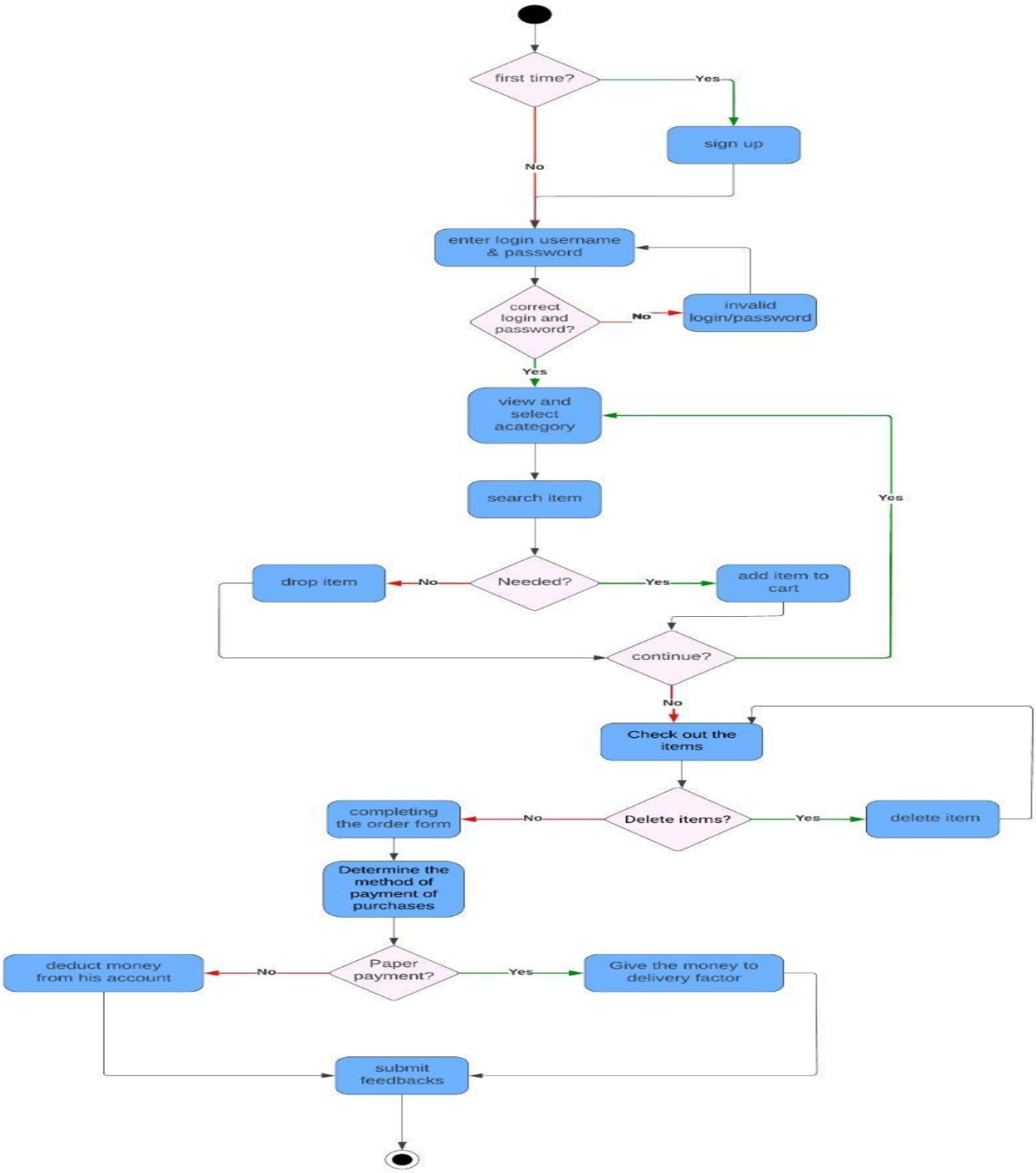
Admin Sequence



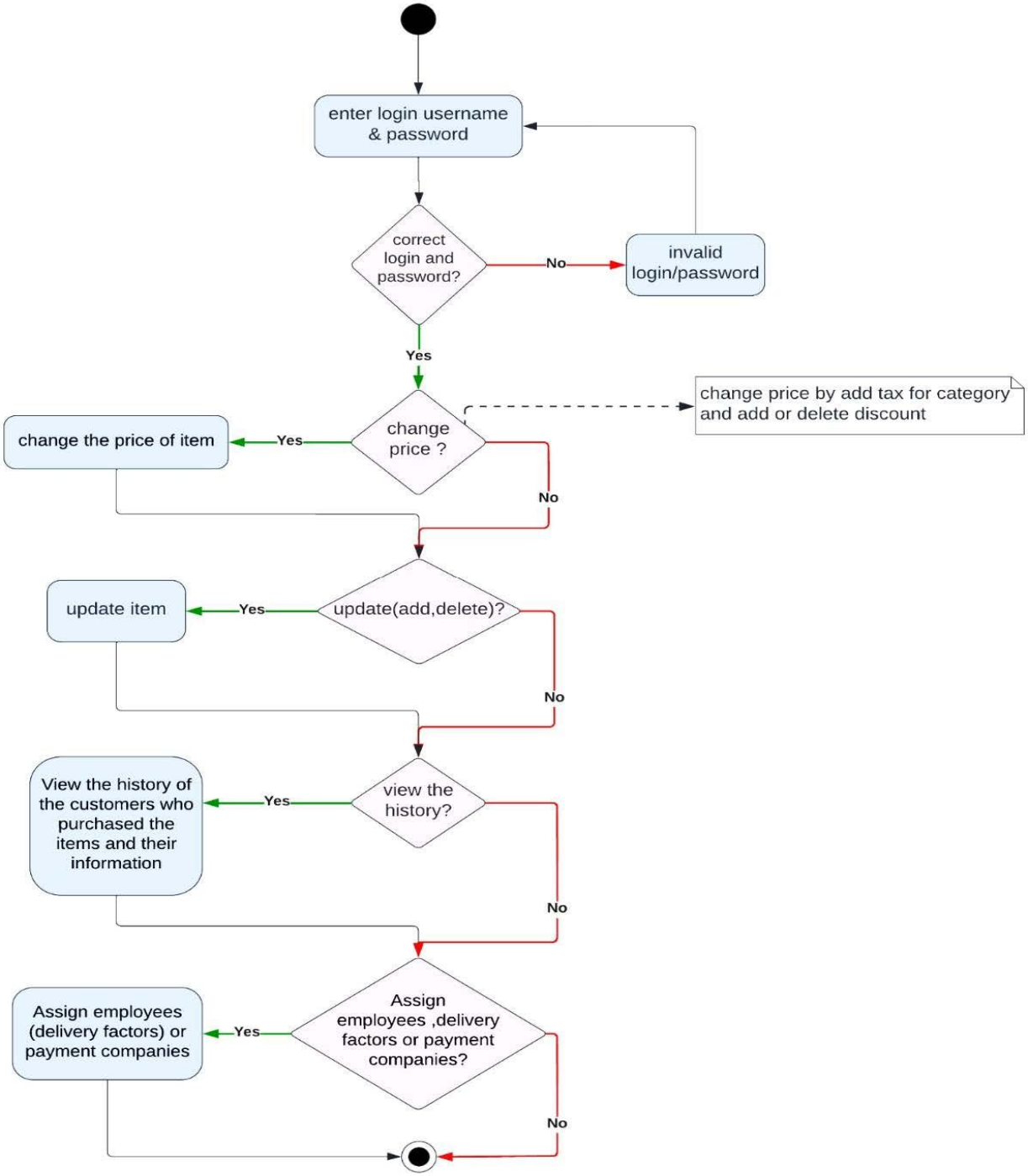
Delivery Agent Sequence



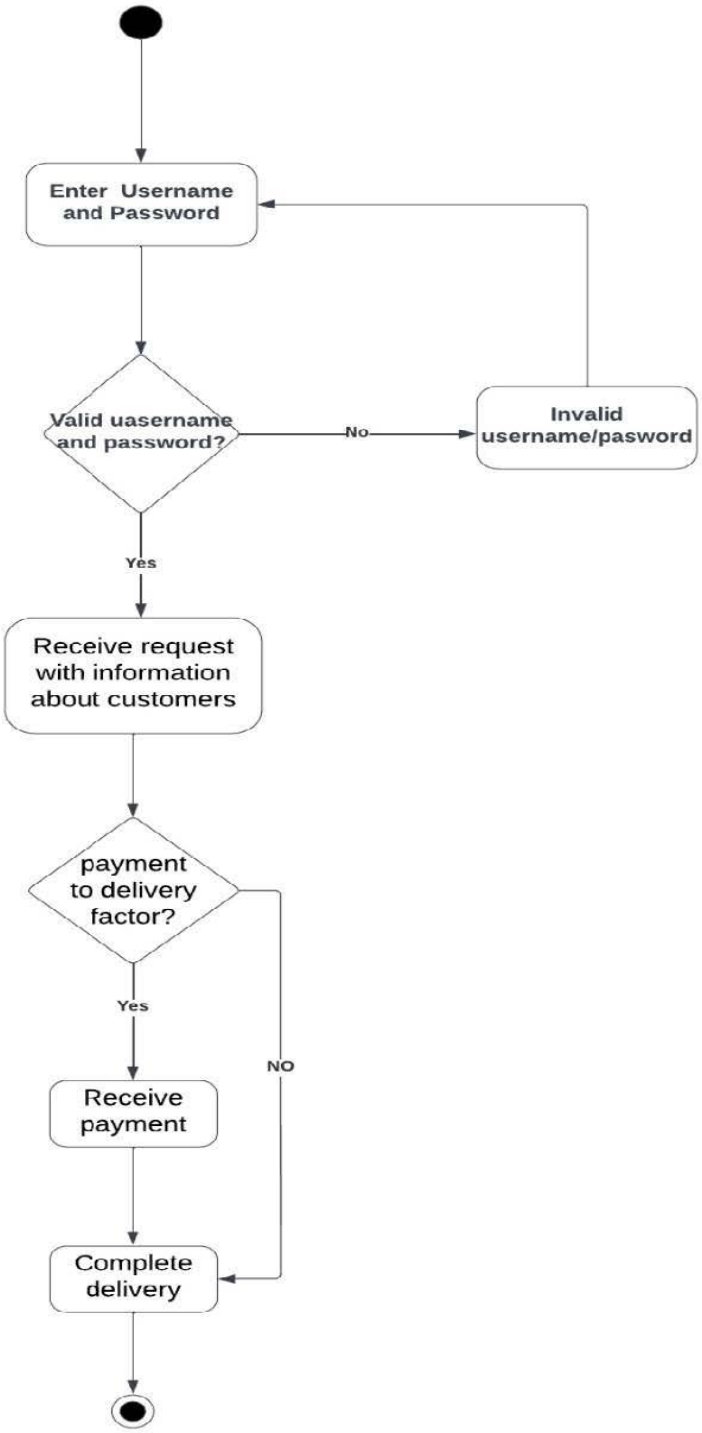
Customer Activity



Admin Activity

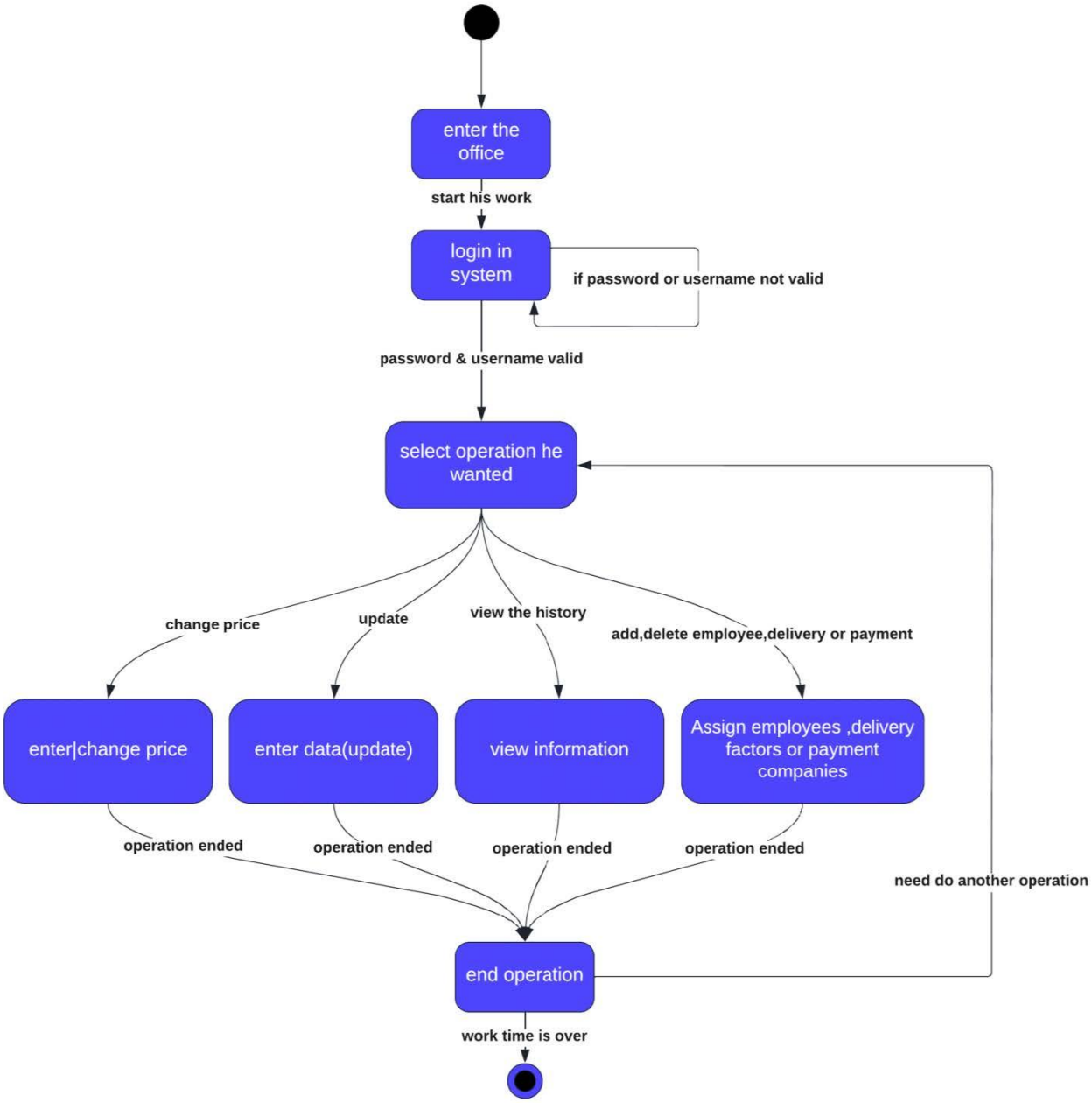


Delivery Agent Activity

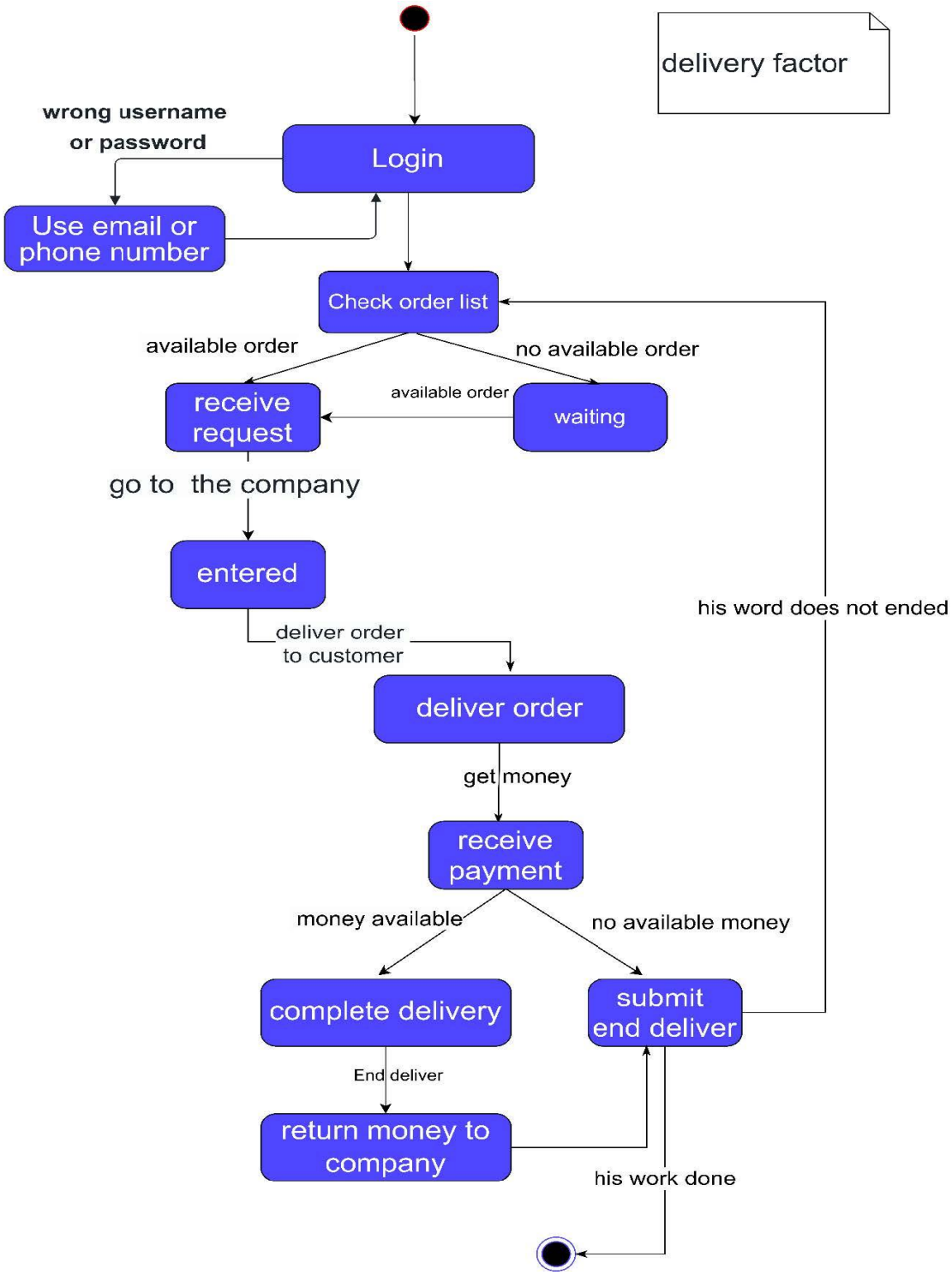


Customer State Diagram

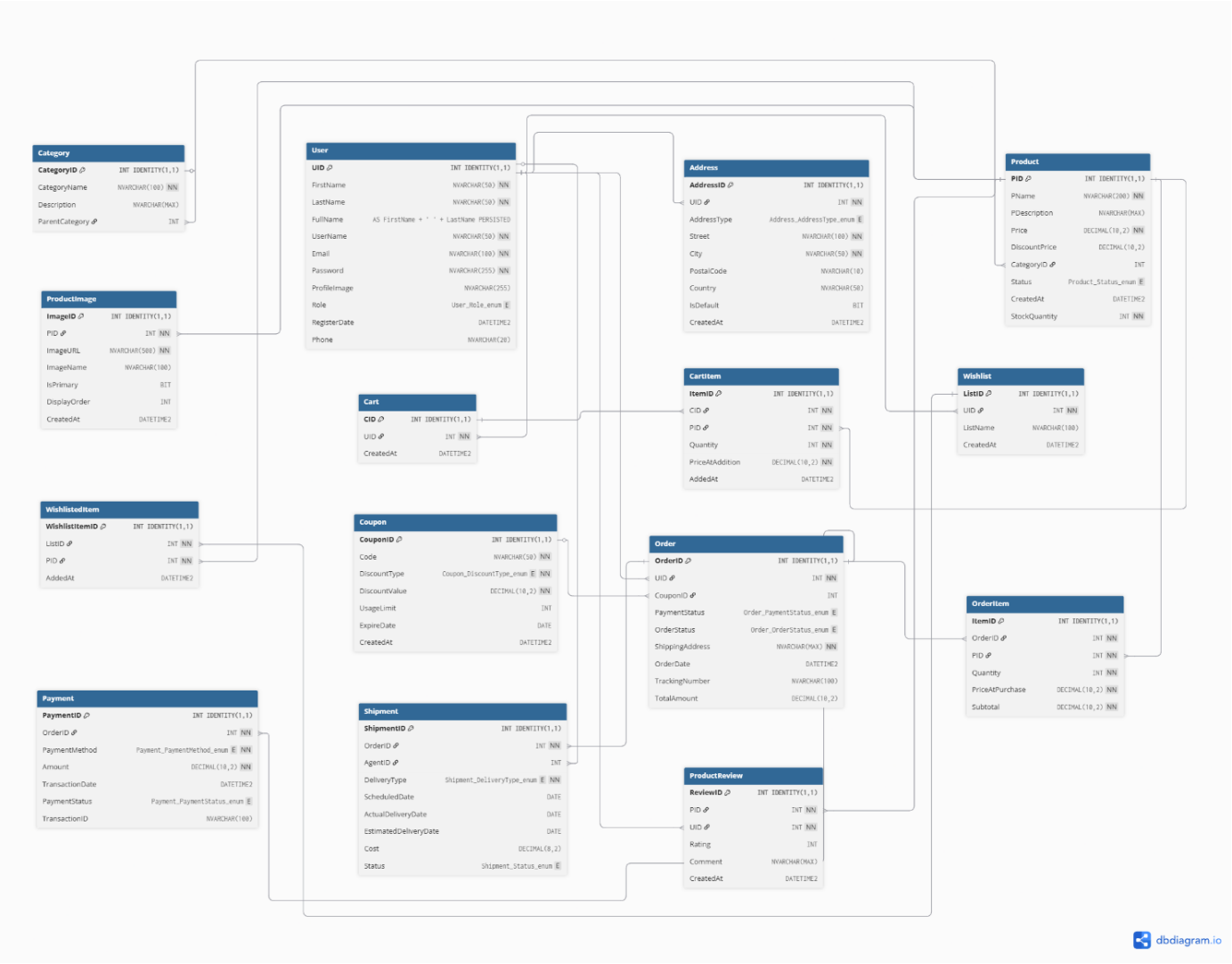
Admin State Diagram



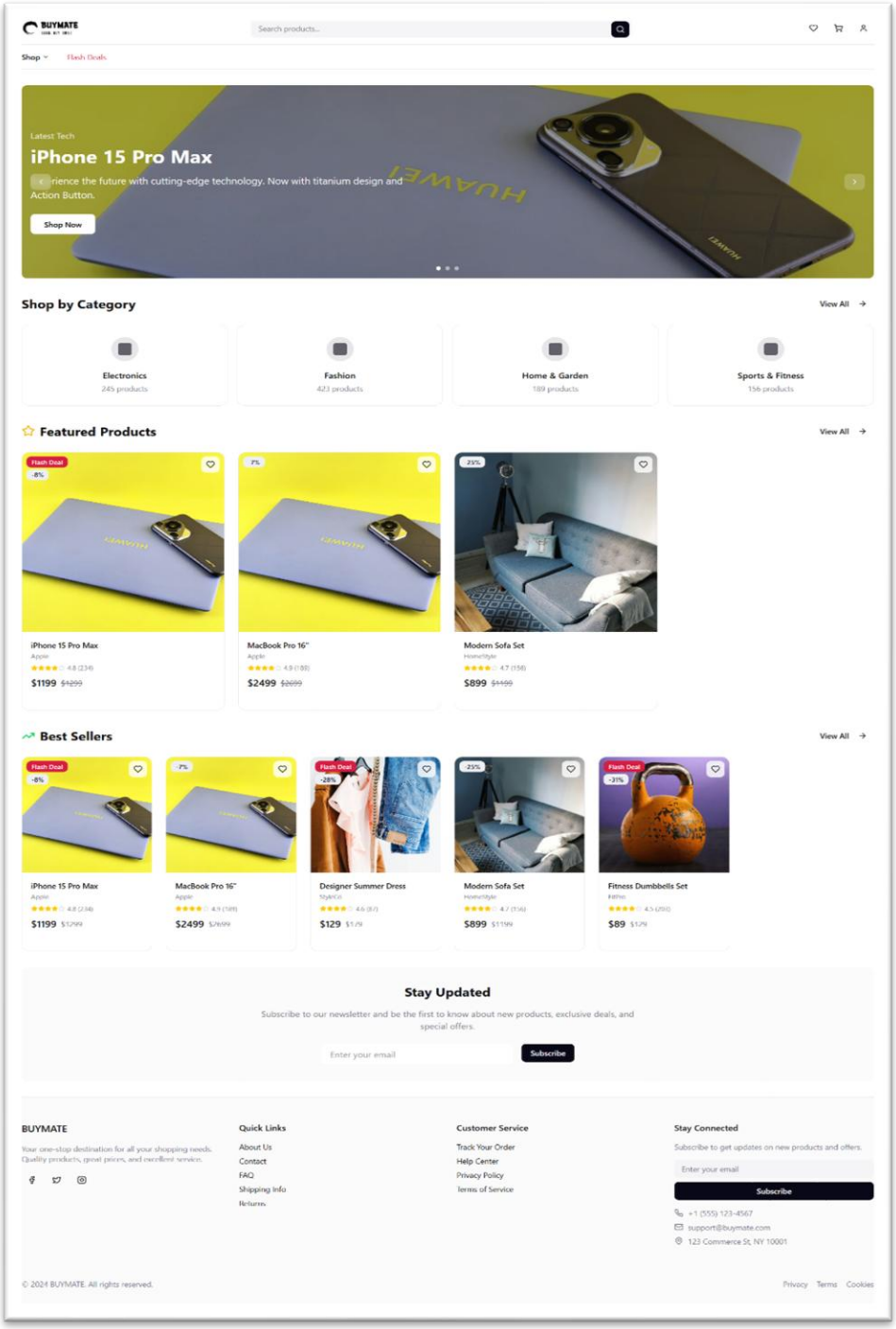
Delivery Agent State Diagram

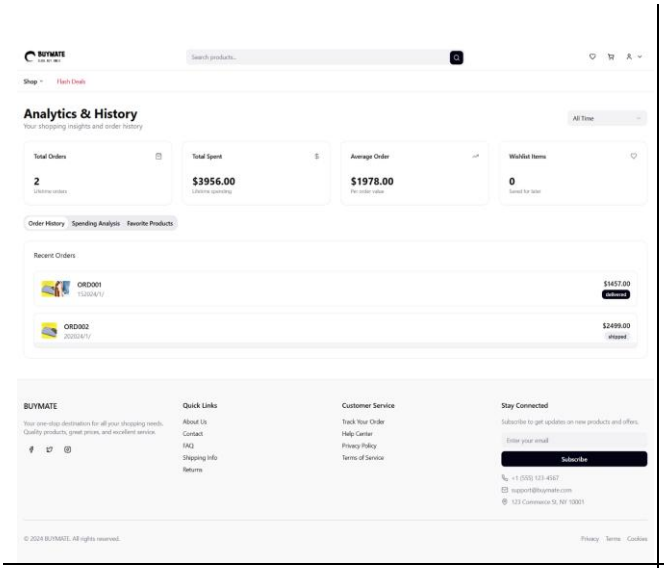


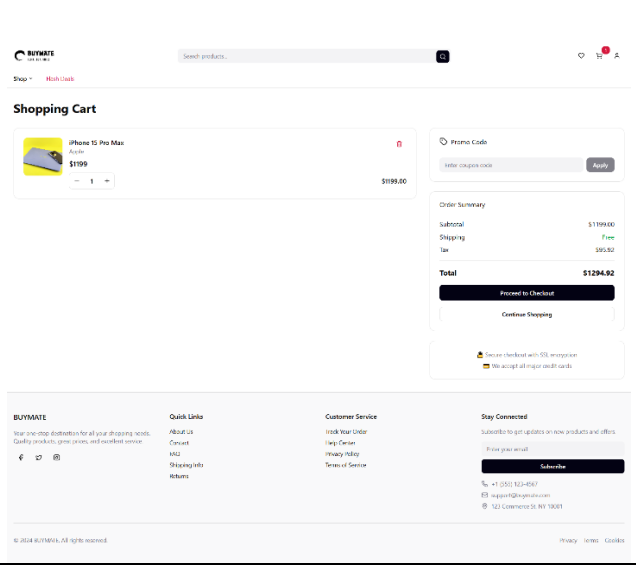
Class Diagram

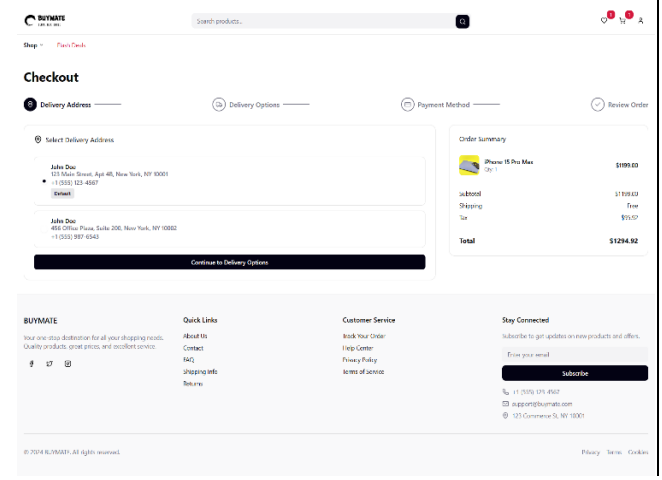


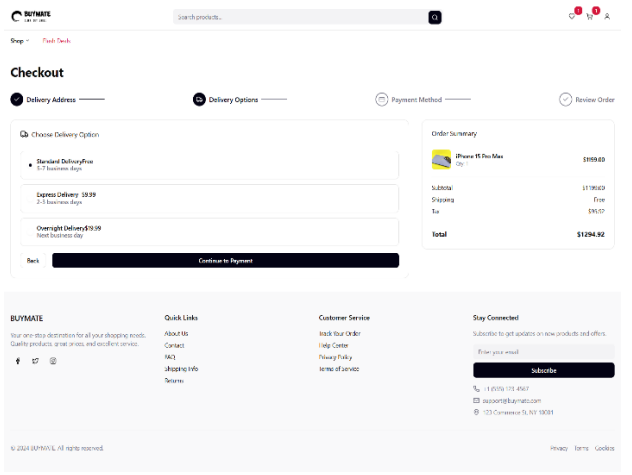
iv. UI/UX Design & Prototyping

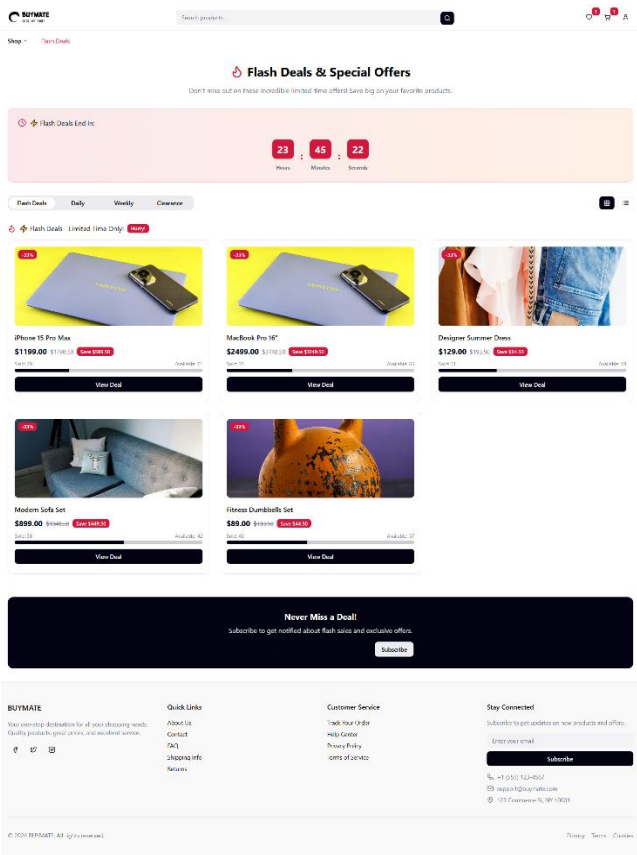


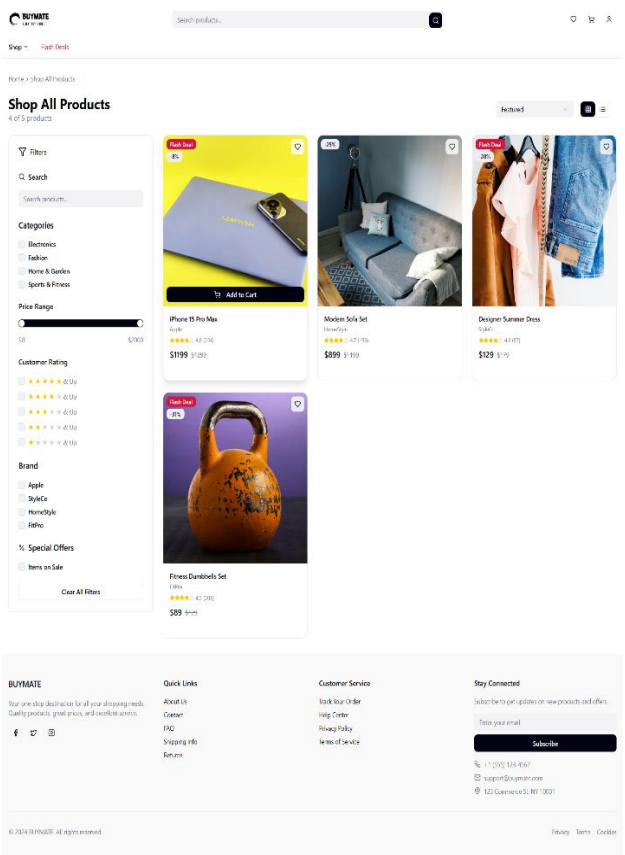

BUYMATE
Shop
Flash Deals
Analytics & History
Your shopping insights and order history
All Time
Total Orders
2
Total Spent
$3956.00
Average Order
$1978.00
Wishlist Items
0
Order History
Spending Analysis
Favorite Products
Recent Orders
ORD001
$1457.00
Delivered
ORD002
$2499.00
Shipped
BUYMATE
Quick Links
About Us
Contact
FAQ
Shipping Info
Returns
Customer Service
Track Your Order
Help Center
Privacy Policy
Terms of Service
Stay Connected
Subscribe
support@buymate.com
123 Commerce St, NY 10001
© 2024 BUYMATE. All rights reserved.
Privacy
Terms
Cookies

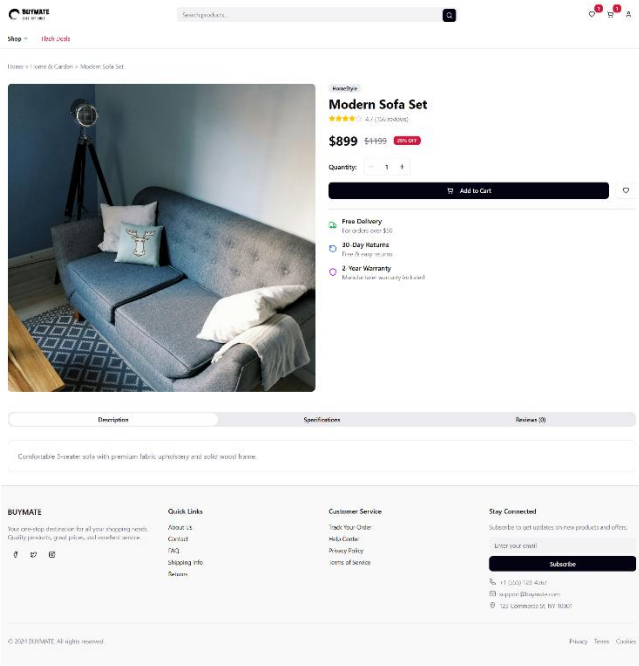

BUYMATE
Shop
Flash Deals
Shopping Cart
iPhone 15 Pro Max
Apple
$1199
$1199.00
Promo Code
Apply
Order Summary
Subtotal
$1199.00
Shipping
Free
Tax
$95.92
Total
$1294.92
Proceed to Checkout
Continue Shopping
Secure checkout with SSL encryption
We accept all major credit cards
BUYMATE
Quick Links
About Us
Contact
FAQ
Shipping Info
Returns
Customer Service
Track Your Order
Help Center
Privacy Policy
Terms of Service
Stay Connected
Subscribe
© 2024 BUYMATE. All rights reserved.
Privacy
Terms
Cookies

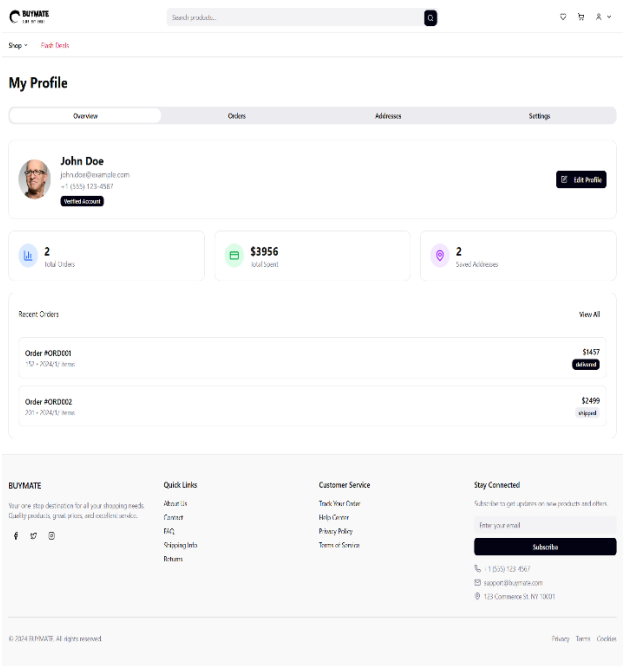

BUYMATE
Shop
Flash Deals
Checkout
Delivery Address
Delivery Options
Payment Method
Review Order
Select Delivery Address
John Doe
123 Main Street, Apt 4B, New York, NY 10001
+1 (555) 123 4567
Default
John Doe
456 Office Plaza, Suite 200, New York, NY 10002
+1 (555) 987 6543
Continue to Delivery Options
Order Summary
iPhone 15 Pro Max
Qty: 1
$1199.00
Subtotal
$1199.00
Shipping
Free
Tax
$95.92
Total
$1294.92
BUYMATE
Quick Links
About Us
Contact
FAQ
Shipping Info
Returns
Customer Service
Track Your Order
Help Center
Privacy Policy
Terms of Service
Stay Connected
Subscribe
© 2024 BUYMATE. All rights reserved.
Privacy
Terms
Cookies


BUYMATE
Shop
Flash Deals
Checkout
Delivery Address
Delivery Options
Payment Method
Review Order
Choose Delivery Option
Standard Delivery Free
5-7 business days
Express Delivery $9.99
2-3 business days
Overnight Delivery $19.99
Next business day
Back
Continue to Payment
Order Summary
iPhone 15 Pro Max
Qty: 1
$1199.00
Subtotal
$1199.00
Shipping
Free
Tax
$95.92
Total
$1294.92
BUYMATE
Quick Links
About Us
Contact
FAQ
Shipping Info
Returns
Customer Service
Track Your Order
Help Center
Privacy Policy
Terms of Service
Stay Connected
Subscribe
© 2024 BUYMATE. All rights reserved.
Privacy
Terms
Cookies


Flash Deals & Special Offers
Never Miss a Deal!
Subscribe


Shop All Products


Modern Sofa Set

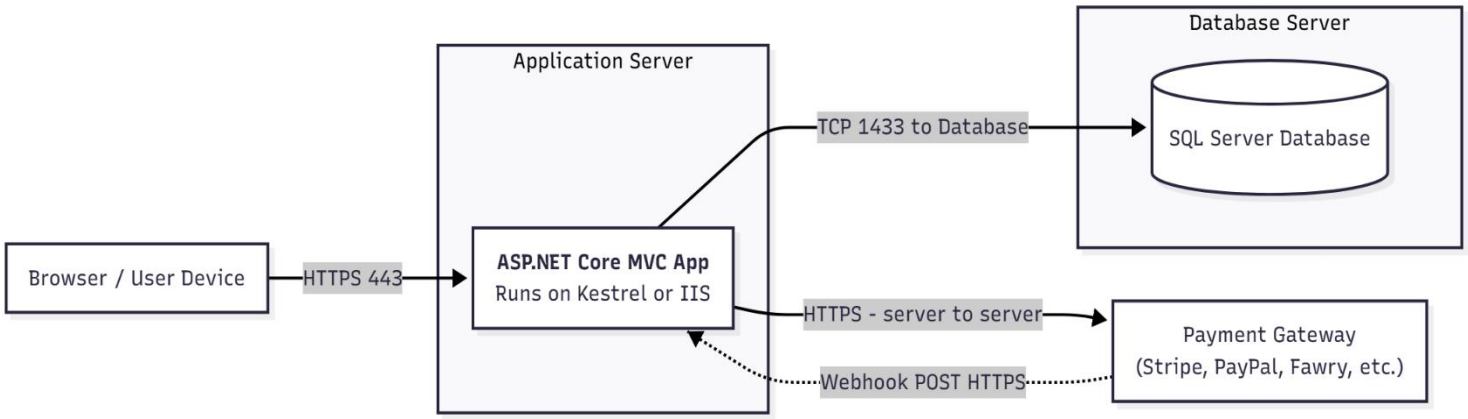

My Profile

v. System Deployment & Integration

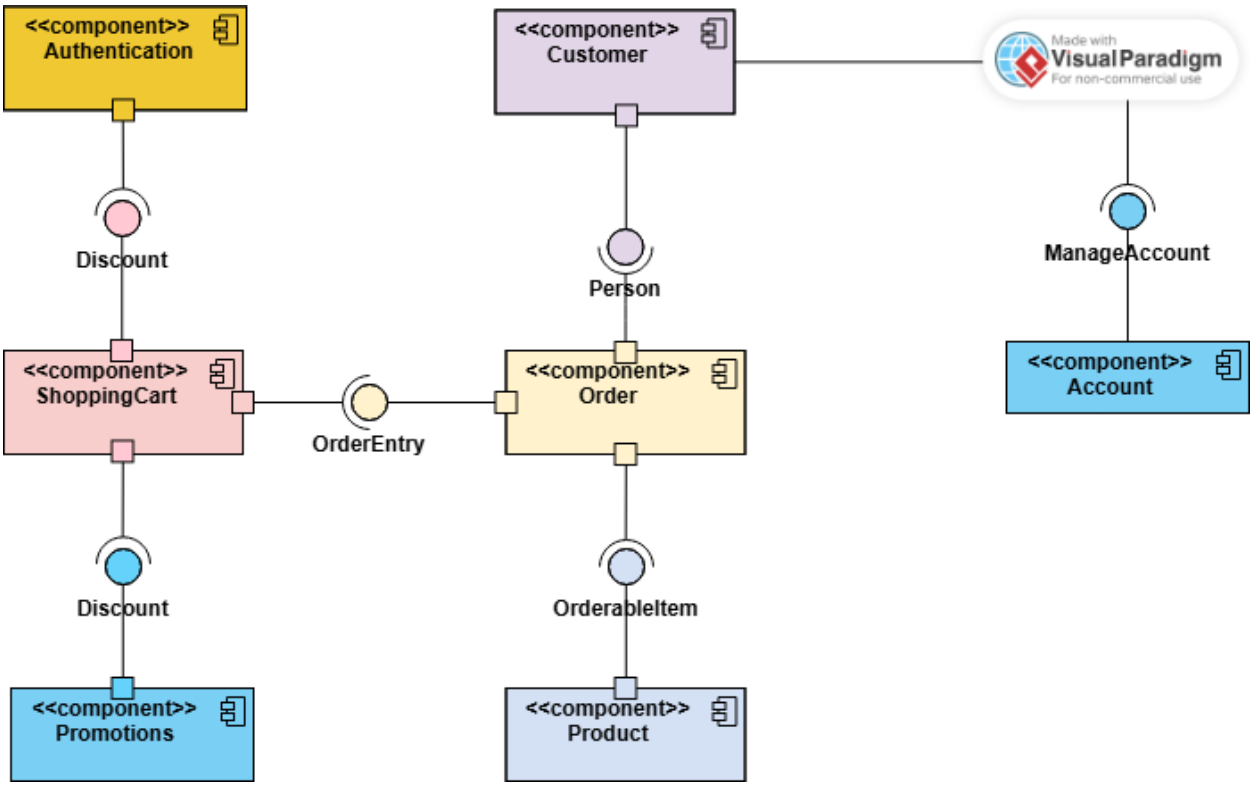
• Technology Stack:

Layer / Concern	Technology	Version / Notes	Purpose
Backend framework	ASP.NET Core MVC	.NET 9	Main application framework, controllers, views, routing
Language	C#	Latest supported by chosen .NET SDK	Business logic and server code
Web server / Host	Kestrel + reverse proxy (Nginx on Linux or IIS on Windows)	Container-friendly	Serve HTTP; TLS termination + reverse proxy for load balancing
Database	Microsoft SQL Server	SQL Server 2022 (Express)	Persistent relational storage
Data access / ORM	Entity Framework Core (SQL Server provider)	+ option: Dapper for performance-critical queries	ORM for DB migrations, queries; Dapper for micro-optimizations
Frontend (server-side)	Razor Views (ASP.NET MVC)	Bootstrap 5 for styling	Server-rendered pages and forms (fast, SEO-friendly)
File / Blob Storage	Azure Blob Storage (or local disk for dev)	Use cloud blob storage in production	Store product images, invoices, backups
Authentication	ASP.NET Core Identity + cookie auth	Social logins via OAuth (Google/Facebook) optional	Manage users, roles, secure sessions
Payments	Stripe, PayPal or Fawry	Use SDKs and HTTPS webhooks	Process payments, handle webhooks securely
Email	SendGrid or SMTP provider	Use transactional email service	Order confirmations, password resets
Logging & Monitoring	Serilog (centralized) + Seq or Azure Application Insights	Aggregated logs and metrics	Error tracking, performance monitoring
Testing	xUnit, Moq (unit), Playwright / Selenium (E2E)	CI integration	Ensure correctness and regressions are caught
Dev tools	Visual Studio / VSCode, SQL Server Management Studio	Developer productivity	Local debugging, DB access

- **Deployment Diagram:**



- **Component Diagram:**



5. Implementation (Source Code & Execution)

1. Source Code

• Structured & Well-Commented Code – Clean, maintainable, and properly documented code following best practices.

• Coding Standards & Naming Conventions – Consistent formatting, meaningful variable names, and adherence to industry standards.

• Modular Code & Reusability – Organized into reusable components, functions, and classes.

• Security & Error Handling – Secure coding practices, validation checks, and proper exception handling.

2. Version Control & Collaboration

• Version Control Repository – Hosted on GitHub, GitLab, or Bitbucket with a public/private repository link.

• Branching Strategy – Clear workflow (e.g., GitFlow, Feature Branching) for managing code updates.

• Commit History & Documentation – Meaningful commit messages and detailed pull request descriptions.

• CI/CD Integration (if applicable) – Automated builds, testing, and deployment pipelines.

3. Deployment & Execution

- README File – Includes:
- Installation steps
- System requirements (hardware/software dependencies)
- Configuration instructions
- Execution guide (running the project locally or accessing a deployed version)
- API documentation (if applicable)
- Executable Files & Deployment Link –
- Compiled software or packaged application (e.g., .exe, .jar, .apk).
- Deployed web/mobile app

6. Testing & Quality Assurance

- **Test Cases & Test Plan** – Document detailing test scenarios and expected outcomes.
- **Automated Testing (if applicable)** – Any automated test scripts used.
- **Bug Reports** – Issues identified and resolutions.

7. Final Presentation & Reports

- **User Manual** – Instructions for end users.
- **Technical Documentation** – System architecture, database schema, API documentation.
- **Project Presentation (PPT/PDF)** – Summary of the project, challenges, solutions, and outcomes.
- **Video Demonstration (Optional)** – Short demo showcasing the project's functionality.